

PCBWORLD: A Benchmark Environment for Engine-Grounded PCB Design Automation

Hyungseok Song*
LG AI Research
Seoul, Republic of Korea
hyungseok.song@lgresearch.ai

Junseok Park*
LG AI Research
Seoul, Republic of Korea
frank.park227@lgresearch.ai

Won-Seok Choi*
LG AI Research
Seoul, Republic of Korea
wonseok.choi@lgresearch.ai

Seohui Bae
LG AI Research
Seoul, Republic of Korea
seohui.bae@lgresearch.ai

Han-Seul Jeong
LG AI Research
Seoul, Republic of Korea
hanseul.jeong@lgresearch.ai

Youngjoon Park[†]
LG AI Research
Seoul, Republic of Korea
yj.park@lgresearch.ai

Soonyoung Lee[†]
LG AI Research
Seoul, Republic of Korea
soonyoung.lee@lgresearch.ai

Abstract

PCB routing is the task of connecting the nets of a board with copper traces under strict design rules, yet learning-based methods still lag behind rule-based routers. We introduce PCBWORLD, an open-source engine-grounded PCB routing environment built on KiCAD, an electronic design automation (EDA) engine. As a human engineer does, agents in PCBWORLD interactively route a board through the engine’s native operations, guided by its Design Rule Check (DRC) feedback. The environment supports both RL and tool-using LLM agents. Alongside the environment, PCBWORLD-BENCH provides three board datasets in the native .kicad_pcb format, two controllable synthetic generators and 679 real open-source boards. It scores any completed board with eight engine-checked evaluation metrics, regardless of the routing method. In our experiments, agents in PCBWORLD consistently outperformed grid-action RL policies and open-loop LLM baselines, and an RL policy trained only on synthetic boards transferred zero-shot to real boards, approaching rule-based routers. PCBWORLD and PCBWORLD-BENCH are available at <https://github.com/LGAI-Research/PCBWORLD>.

Keywords

Agentic AI, PCB Routing Benchmark, Reinforcement Learning

1 Introduction

Printed circuit boards (PCBs) are the physical boards that mount and interconnect electronic components, forming the backbone of nearly every electronic product [10, 21, 23]. Turning a circuit design into a manufacturable board centers on *routing*, the task of drawing copper traces that connect the pads of each net while keeping distinct nets electrically isolated under strict design rules [50]. Yet industry still relies mainly on rule-based routers rooted in decades-old heuristics [12, 26, 34], which automate much of the routing process but rarely complete complex production boards end-to-end. Learning-based methods have been studied extensively to overcome

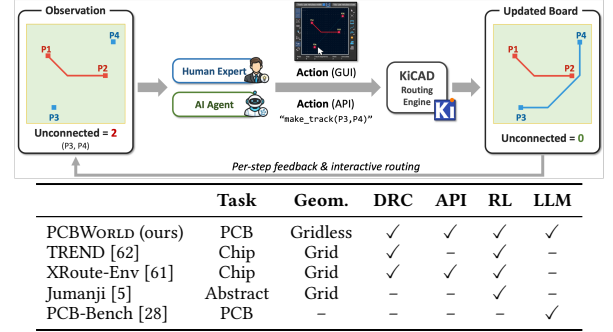


Figure 1: Overview of PCBWORLD and PCBWORLD-BENCH. Agents invoke the KiCAD engine’s native API and condition each step on engine-computed feedback.

this limit [5, 38], but unlike in language, vision, and games, they are not yet competitive even with rule-based routers.

We attribute this limitation to how the routing problem is modeled. Existing RL formulations either cast routing as cell-by-cell movement on a grid, where the search space grows rapidly with grid size [5, 38], or delegate routing to an off-the-shelf router, steering it through auxiliary decisions such as net ordering [30, 61, 62]. Recent LLM-based methods generate electronic design automation (EDA) scripts [54], Verilog modules [4], or schematics [63] as whole artifacts, revising them at most on per-artifact verdicts. None targets board routing, where the strict design rules bind. The workflow that reliably completes production boards remains that of a human engineer, who neither hands the board to an auto-router nor draws every trace by hand. Instead, the engineer repeatedly reads the current board, judges where the next connection can feasibly run, and draws it with the engine’s native operations, which keep the trace within the design rules.

To place agents in this same loop, we introduce PCBWORLD, an *engine-grounded* PCB environment in which agents complete boards

*Equal contribution. [†]Corresponding author.

by directly invoking an EDA engine’s native routing operations. Built on the open-source KiCAD EDA engine [25, 35], PCBWORLD exposes 58 Python APIs over the engine, from step-level routing operations to its Design Rule Check (DRC). The agent routes the board itself in a closed loop, receiving the updated board state and DRC feedback after every operation. On top of a standard Gym interface, PCBWORLD provides tailored wrappers for RL policies and tool-using LLM agents, so both families train and act in the same environment. Unlike GUI-driven agent environments [56], PCBWORLD also runs headless and vectorized, sustaining the throughput that large-scale RL training requires.

Alongside PCBWORLD, we release PCBWORLD-BENCH, a benchmark that evaluates *geometric feasibility reasoning*, the capability to produce a routed board in which every net is connected and every trace satisfies the strict design rules. This capability remains largely untested in existing LLM benchmarks, which focus on software engineering [22], board interpretation [28], or 3D-CAD generation [52, 55]. PCBWORLD-BENCH combines two synthetic board generators, one grid-based (D1) and one gridless (D2), with a curated set of 679 real open-source boards (D3). Every instance is provided in KiCAD’s native board format (.kicad_pcb) with its own design rules, and loads directly into PCBWORLD for training and evaluation. The evaluation is method-agnostic, scoring any completed board with the same engine whether or not the method routes through PCBWORLD.

On PCBWORLD-BENCH, we evaluated three families of routing methods under the same protocol: rule-based routers, RL agents, and LLM agents. Three findings emerge. (i) Agents acting through PCBWORLD outperform their counterparts, with PPO maintaining performance at grid resolutions where grid-action RL collapses and interactive LLM routing beating the open-loop baselines. (ii) Trained only on synthetic boards, PPO transfers zero-shot to real boards, outperforming all baselines on small boards while trailing the strongest rule-based router on larger ones. (iii) Large-board routing remains an open challenge for both RL and LLM agents. We release PCBWORLD and PCBWORLD-BENCH as open source, as a shared foundation for evaluating and advancing learning-based PCB routing.

1.1 PCB Routing Problem

A printed circuit board (PCB) is a multi-layer substrate whose components expose metal contacts called *pads* (Figure 2). A *net* is a set of pads that must be electrically connected to share the same signal, and PCB routing is the task of drawing copper *traces* that connect all pads of every net while keeping distinct nets electrically isolated. A trace is represented as a sequence of straight *track* segments, each lying on a single layer. A routing scheme is *grid-based* if track endpoints must lie on a fixed lattice, and *gridless* if they may take arbitrary positions on the canvas. A trace may switch layers through a *via*, a plated hole that joins tracks across layers. A routed board is valid only if it passes a *Design Rule Check* (DRC). Any *design rule violation* (DRV), for example a *clearance* violation between distinct nets, blocks fabrication. We call a DRV-free routing *geometrically feasible*. A single violation invalidates an otherwise plausible board.

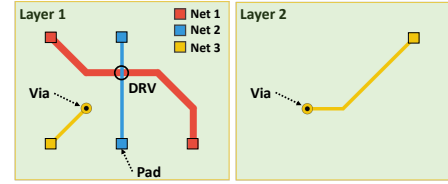


Figure 2: Core concepts of PCB routing. Pads of the same net must be electrically connected. Traces of different nets that come too close incur a design rule violation (DRV). Vias let traces switch layers to avoid such conflicts.

2 Background

Among DRV-free routings, quality is measured by total *wirelength* and *via count*, which correlate with signal delay and fabrication cost. For the routing state s , let $n_{\text{drv}}(s)$, $\ell(s)$, and $n_{\text{via}}(s)$ denote the number of DRVs, the total wirelength, and the via count, respectively. The PCB routing task is then formulated as a constrained optimization problem:

$$s^* = \arg \min_{s \in \mathcal{R}} \lambda_w \ell(s) + \lambda_v n_{\text{via}}(s) \quad \text{subject to} \quad n_{\text{drv}}(s) = 0, \quad (1)$$

where $\mathcal{R}$ is the set of board states and $\lambda_w, \lambda_v \geq 0$ are user-specified weights balancing wirelength against via count.

2.1 KiCAD: An Open, Programmable EDA Suite

KiCAD [25] is an open-source EDA suite maintained under the Linux Foundation [35]. It provides the end-to-end PCB design flow within a unified engine, from component placement and trace routing to design rule checking. The C++ engine supports exposing its native operations to Python through extensible bindings [24] and stores boards in an open, human-readable format. The broader ecosystem includes open routing resources such as PCBench [17] and the routers Freerouting [12], OrthoRoute [3], and KiCadRoutingTools [15]. These properties make KiCAD a production-grade open-source alternative to commercial EDA suites.

3 PCBWORLD: An Engine-Grounded PCB Routing Environment

PCBWORLD wraps KiCAD engine as a Gym environment that faithfully reproduces a real-world PCB routing workflow. The environment is organized as three layers: the C++ KiCAD engine performing the actual routing and design rule checks (§3.1), the Gym environment defining an MDP over the engine’s native operations (§3.2), and two tailored wrappers re-encoding this MDP for RL and LLM agents (§3.3).

3.1 Python API via Bindings to the KiCAD Engine

The bottom layer provides 58 *Python APIs* built on bindings to the KiCAD engine (§2.1): 14 reproduce the core routing API of PNS: :ROUTER, 28 directly wrap KiCAD’s DRC and board-state extraction APIs, and 16 provide auxiliary utilities.¹ All higher layers

¹The full list is provided in Appendix C. PCBWORLD is built against KiCAD 9.0.8 with kicad-python 0.6.0.

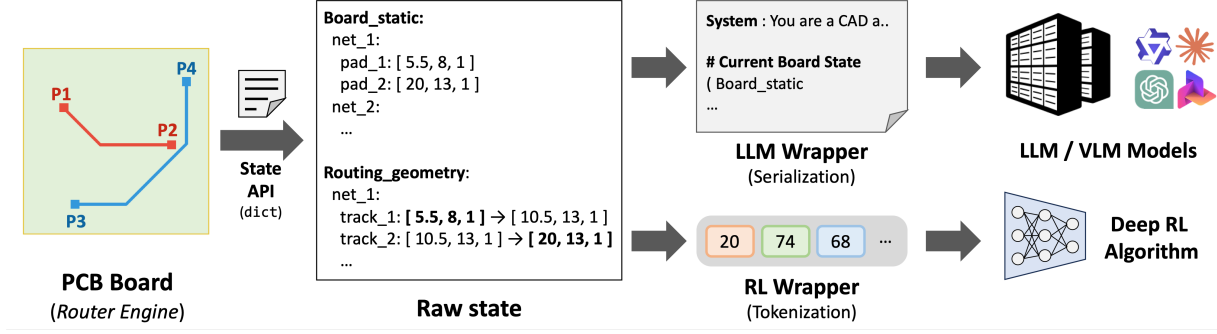


Figure 3: Gym Wrappers. Two Gym wrappers communicate with the shared engine through a unified state dictionary, which they re-encode into agent-specific formats: tokenized sequences for the RL and serialized tool-call schemas for the LLM.

reach the engine only through these headless APIs, so every agent action executes as a native KiCAD engine operation, as in a human engineer’s workflow (Figure 1). Because each environment holds its own engine instance, PCBWORLD also runs vectorized, stepping many boards in parallel.

3.2 MDP Formulation

The Gym environment formalizes the routing workflow as an MDP. An episode loads a real PCB file (.kicad_pcb) as its initial state, a *bare board* whose components are fixed and whose nets are unrouted. The agent then routes the board through successive API calls, observing the result of each, until every net is routed or a step limit is reached.

State. At each time step $t \in \{0, 1, \dots, T\}$, the state s_t captures a snapshot of the board, where $t = 0$ is the bare board and $t = T$ the terminal state. A KiCAD board is naturally described as a composition of heterogeneous geometric objects organized in a hierarchical structure (e.g., $\text{net} \rightarrow \text{pad} \rightarrow \text{coordinate}$). We introduce a **nested dictionary** that faithfully represents the exact numeric values and hierarchy of KiCAD (Figure 3). Unlike abstracted representations such as a grid or a rendered image, this lets the agent observe the board’s exact state, as the EDA domain’s strict design rules demand. The dictionary splits by mutability during routing. **Board_static** holds immutable objects such as pad geometries, internal obstacles, and design rules. **Routing_geometry** holds objects added during routing, such as tracks, vias, and the per-pad connectivity status. Further details are provided in Appendix E.

Action. An action is a pair of an action type and its arguments, composed from the APIs exposed in §3.1. Table 1 summarizes the six action types. In the example of Figure 4, `make_line` takes arguments (`p_end`, `routing_mode`) and draws a track to `p_end`. The `routing_mode` argument controls how the engine realizes this track, either pushing blocking wires aside (`push_n_shove`) or detouring around them (`walkaround`), while satisfying strict design rules such as clearance. The KiCAD engine exposes a finite set of candidate points for `p_end`, such as pad centers and the endpoints of existing tracks and vias. This discretizes the continuous plane into a small candidate set, as an engineer’s editor does, and the agent selects a `p_end` and a `routing_mode` rather than drawing raw segments itself. The

Table 1: Routing actions.

Action	Arguments	Description
<code>net_select</code>	<code>net_id</code>	Select <code>net_id</code> for subsequent tracks and vias.
<code>net_end</code>	–	Release the current net.
<code>start_route</code>	<code>p_start</code>	Begin a route from <code>p_start</code> , a pad center or existing track/via endpoint of the net.
<code>make_line</code>	<code>p_end</code> , <code>routing_mode</code>	Extend the route to <code>p_end</code> on the current layer. <code>routing_mode</code> sets how it interacts with existing traces.
<code>make_via</code>	<code>p_end</code> , <code>routing_mode</code>	Execute <code>make_line</code> to <code>p_end</code> , then place a via at <code>p_end</code> .
<code>finish</code>	<code>routing_mode</code>	Auto-complete the route to the Euclidean-nearest unconnected pad on the current layer.

RL agent selects `p_end` from this candidate set. The LLM agent is prompted toward the same points but may emit free coordinates for detour waypoints.

Reward. We relax Equation (1)’s hard DRV constraint into a penalty and define a potential function $\Phi(s)$ that measures the board quality,

$$\Phi(s) = -(f_{\text{drv}}(s) + \lambda_w \ell(s) + \lambda_v n_{\text{via}}(s)).$$

The DRV count $n_{\text{drv}}(s)$ comes from KiCAD’s DRC API, which evaluates violations across 38 checks. The two weights $\lambda_w, \lambda_v \geq 0$ reuse those of Equation (1), and $f_{\text{drv}}(s)$ is a concave penalty that increases with every violation and is steepest near zero DRVs. This penalty dominates the wirelength and via terms, so maximizing Φ drives the policy toward DRV-free routings that approximate the optimum of Equation (1). The full check catalog is given in Appendix F, and the exact form of f_{drv} in Appendix G.

The MDP’s reward is the *terminal* reward, with $r_t = 0$ for $t < T$ and $r_T = \Phi(s_T) - \Phi(s_0)$ at termination, scoring the completed board against the bare board. For dense training we also expose a *per-step* form, $r_t = \Phi(s_{t+1}) - \Phi(s_t)$. With an undiscounted return ($\gamma = 1$), the per-step rewards telescope to $\sum_{t=0}^{T-1} r_t = \Phi(s_T) - \Phi(s_0)$, so the per-step form is a potential-based shaping of the terminal reward [40] and shares its optimal policy. Since GRPO [48] uses

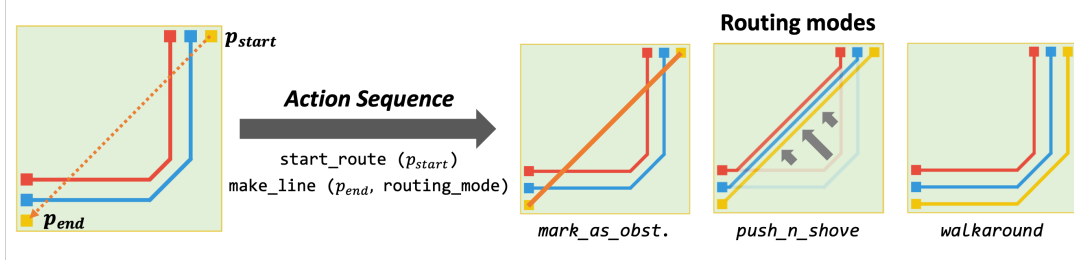


Figure 4: Example Action of PCBWORLD. An example of `make_line`, where the `routing_mode` argument alters routing behavior through the KiCAD routing engine, yielding different trace geometries across the three modes.

only the total episode return, the two reward forms coincide for it. For PPO [47], which estimates advantages with a value function, the per-step form gives a denser credit-assignment signal. Because the equivalence holds for any potential, PCBWORLD exposes Φ as a pluggable training objective. Further details are provided in Appendix K.2.

3.3 Environment Wrappers

We implement separate wrappers that expose the nested-dictionary raw state and structured actions through interfaces tailored to RL and LLM tasks. The RL wrapper tokenizes each geometric object (pad, track, via, point) into an individual token by combining a Fourier feature map of its coordinates with an entity-type embedding, yielding a flat sequence that preserves the native object hierarchy. The action is decoded autoregressively: the head emits an action type, then only the parameter tokens that type requires. Depending on the type, these are a point selected by a pointer network from the engine-provided candidate set, a routing mode, or both. This keeps every emitted action syntactically complete and within the engine’s valid-action set by construction.

For the LLM wrapper, we adopt an S-expression representation that preserves the nested hierarchy and key-value relations while minimizing token count, retaining only routing-relevant fields (coordinates, connectivity, layers, effective DRC constraints). The prompt distills the task into a concise set of routing guidelines. Finally, the wrapper supplies a compact action history that flags the most recent rejected or no-effect action, so the agent avoids repeating it. Full details of both wrappers are provided in Appendix H.

4 PCBWORLD-BENCH

We introduce PCBWORLD-BENCH, a benchmark for geometric feasibility reasoning in PCB routing. It provides a method-agnostic evaluation protocol, three board datasets, and eight evaluation metrics.

4.1 Task and Evaluation Protocol

Each instance is a KiCAD board in the native `.kicad_pcb` format with its routes removed. Given the component placement and design rules, the agent completes the routing in the same file. The PCBWORLD-BENCH evaluator scores the completed board with the KiCAD engine. Since the evaluation depends only on the board,

Table 2: Dataset statistics. The table reports the number of boards and the per-board ranges of net, pad, and layer counts.

Dataset	# inst. (train/valid/test)	# nets	# pads	# layers
D1 (Grid)	10,000 / 128 / 128	5	10	1
D2 (Gridless)	10,000 / 128 / 128	4–6	8–21	2
D3-A (Small)	– / – / 100	2–13	6–31	2–4
D3-B (Medium)	– / – / 287	5–42	31–100	2–4
D3-C (Large)	– / – / 292	6–451	101–2,103	2–8

the protocol is method-agnostic. Routing methods that do not use PCBWORLD and the reference solutions are scored the same way as our agents.

4.2 Board Datasets

Table 2 summarizes the instance counts and key statistics of the three board datasets, and Figure 5 presents representative examples from each. The datasheet and license information are provided in Appendices O and P.

(D1) *Synthetic Grid-based Boards*. D1 replicates the grid-routing task of Jumanji’s *Connector* environment [5] in PCBWORLD. Boards are generated in KiCAD format from the Connector instance distribution, and agents route on the same grid, one wire per cell. Five grid sizes are provided, $G \in \{10, 50, 100, 200, 500\}$, denoted D1-10 through D1-500. D1 enables direct comparison with state-of-the-art methods developed on Connector, such as Sable [38].

(D2) *Synthetic Gridless Boards*. D2 is a synthetic gridless routing dataset for training and in-distribution evaluation. We release its board generator, which controls the number of nets and pads per net and thus adjusts routing difficulty directly.

(D3) *Open-Source Boards*. D3 is a curated set of 679 boards derived from PCBench’s open-source PCB corpus [17]. We convert these boards to the KiCAD 9 format, preserving their design-rule settings, and retain only boards that remain fully connected and DRV-free after conversion. Each board retains its original routed traces, which serve as reference solutions for evaluation. These boards are partitioned by pad count into D3-A/B/C, from smallest to largest. For zero-shot evaluation, all methods share a common set of 99 D3-A boards and 10 D3-B boards. Owing to the high per-board cost of LLM evaluation, the D3-B subset is capped at ten boards,

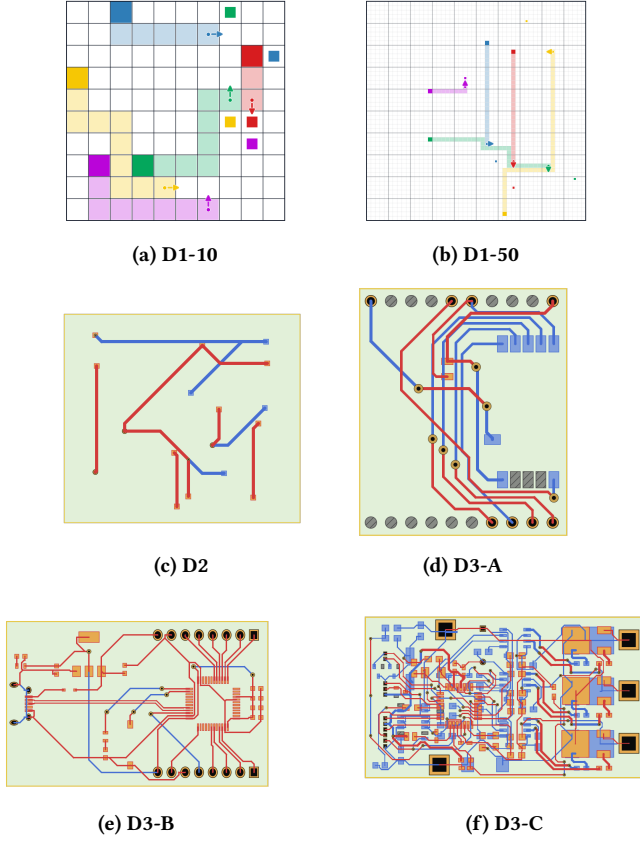


Figure 5: Example boards from PCBWORLD-BENCH. Synthetic grid-based boards (D1), synthetic gridless boards (D2), and real-world open-source boards (D3).

sampled to represent the split’s size range (Appendix O.5). We reserve D3-C, whose boards reach hundreds of nets, as a challenge split for future research.

4.3 Evaluation Metrics

We use **Clean Pass** (CP) as the primary metric. CP counts a board as successful only if the selected rollout achieves full connectivity with zero DRVs. We further report **Potential Gain** (Pot.) as a routing-quality metric, defined as the change in board potential from the bare-board state to the completed routed state. Additional diagnostics include **Routability** (Rout.), the fraction of target connections successfully routed; **wallclock time** (Time); **DRV count** (DRV); and physical cost metrics such as **total wirelength** (WL) and **via count** (Via). For LLM agents we also report **Parse-fail**, the fraction of boards for which no rollout loads as a valid .kicad_pcb, i.e., whose outputs break the KiCAD file format.

For each board we draw five rollouts and select the single rollout with the largest potential gain. All metrics are computed on this selected rollout, except Time, which is averaged over all five rollouts, and Parse-fail, which is computed from all five. We write @5 for this best-of-five protocol, the default for all reported metrics, and

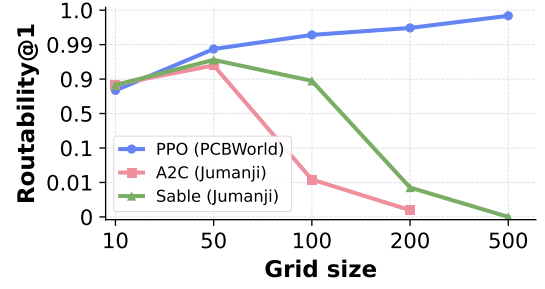


Figure 6: Simulation Results of Synthetic Grid-based Boards (D1). Single-rollout routability (Rout.@1) across grid sizes. The vertical axis is warped symmetrically about 0.5 to make both ends of the range legible. A2C (Jumanji) exhausts memory at D1-500, so its curve stops at D1-200.

@1 when a metric scores a single rollout without selection. Deterministic methods are run once, as selection does not apply. Formal definitions and implementation details are provided in Appendix I.

5 Experiments

We evaluate on the three task suites of §4 (D1, D2, and D3). Through D1, we show that the KiCAD-API actions of PCBWORLD scale to fine grid resolutions where grid-action methods break down. Through D2 and D3, we evaluate the geometric feasibility reasoning of the LLM and RL agents in PCBWORLD against diverse baselines. The RL agents are trained on the D2-train split, while the LLM agents are used off-the-shelf without task-specific fine-tuning.

5.1 Experimental Setup

PCBWORLD agents. We build LLM and RL agents that act in a closed loop over the full PCBWORLD MDP, the *interactive* mode throughout. For **LLM agents**, we examine how routing performance varies with model capacity across GPT-5.4, GPT-5.4-mini, GPT-5.4-nano [41, 42], and Qwen3.5-397B [46]. For **RL agents**, we train a small Transformer policy from scratch in PCBWORLD and report three variants, PPO (per-step), PPO (terminal), and GRPO, using the per-step and terminal rewards defined in §3.2. Since PPO (per-step) is our default policy, we write it as plain PPO unless noted otherwise. We additionally train PPO (w/o finish), removing finish from the action space (§5.5). Further training details are provided in Appendix K.2.

Baselines. We evaluate three baseline families and score the .kicad_pcb files they produce under the same metrics. The **rule-based routers** are Freerouting [12], OrthoRoute [3], and KiCadRoutingTools (KRT) [15]. The **grid-action RL baselines**, A2C (Jumanji) [5] and Sable [38], are trained on Jumanji-Connector v2 and route each net cell by cell on the grid. The two **open-loop LLM baselines** produce their full routing in a single pass, with no feedback on any intermediate board state and no revision on the engine’s verdict: *plan-and-execute* emits the full sequence of PCBWORLD routing actions and has it executed in PCBWORLD, mirroring command-sequence generation in 3D CAD [44, 53], and

Table 3: Routing quality on D2, D3-A, and D3-B. Results follow the @5 protocol of §4.3. We report CP $\uparrow$, Pot. $\uparrow$, Rout. $\uparrow$, and Time $\downarrow$ in seconds. Best CP and Pot. per split (excluding the Reference row) are in bold. Freerouting and the RL agents report the mean over 4 seeds. Per-metric breakdowns and @1 results are provided in Appendix L.

Method	D2				D3-A (99 Boards)				D3-B (10 Boards)			
	CP $\uparrow$	Pot. $\uparrow$	Rout. $\uparrow$	Time $\downarrow$	CP $\uparrow$	Pot. $\uparrow$	Rout. $\uparrow$	Time $\downarrow$	CP $\uparrow$	Pot. $\uparrow$	Rout. $\uparrow$	Time $\downarrow$
Reference	-	-	-	-	1.00	23.16	1.00	-	1.00	63.96	1.00	-
Freerouting	1.00	15.47	1.00	2.69	0.80	22.71	0.91	7.09	0.78	61.06	1.00	9.94
OrthoRoute	0.01	0.38	0.34	2.54	0.02	-6.16	0.53	2.20	0.00	-10.25	0.44	9.30
KiCadRoutingTools	1.00	15.52	1.00	0.82	0.74	20.00	0.94	0.65	0.20	40.94	0.86	3.27
<i>PCBWORLD: Inference on different LLM backbones</i>												
GPT-5.4	0.96	16.05	1.00	94.04	0.65	19.42	0.91	231.15	0.00	34.72	0.62	865.83
GPT-5.4-mini	0.58	12.45	0.86	33.93	0.28	14.22	0.72	56.85	0.00	30.84	0.61	422.06
GPT-5.4-nano	0.55	12.23	0.85	63.39	0.30	14.16	0.73	100.76	0.00	30.66	0.59	510.25
Qwen3.5-397B	0.33	9.77	0.74	47.58	0.34	14.88	0.75	81.82	0.00	25.99	0.51	222.29
<i>PCBWORLD: RL trained on Transformer</i>												
PPO	1.00	16.24	1.00	0.43	0.86	21.46	0.95	1.83	0.45	50.20	0.85	10.77
GRPO	1.00	14.03	1.00	1.16	0.85	18.83	0.98	3.54	0.10	30.00	0.69	11.20
PPO (terminal)	0.92	15.61	0.98	1.37	0.82	21.02	0.95	2.58	0.38	44.33	0.81	12.60
PPO (w/o finish)	1.00	16.21	1.00	0.88	0.94	21.78	0.99	3.05	0.42	46.45	0.87	14.44

engine-free generation directly emits the routing as segments and vias in .kicad_pcb syntax, analogous to one-shot code generation [2, 29, 37].

5.2 Scalability Limits of Grid-Action Routing

On D1, we trained and evaluated each method independently at each grid resolution, training PPO to maximize Rout.@1 to match the objective of A2C (Jumanji) and Sable (Figure 6). On D1-10, PPO is marginally below the grid-action baselines. It reaches near-perfect routability from D1-50 onward, while A2C collapses by D1-100 and Sable by D1-200. As the grid grows finer (Figures 5a and 5b), grid-action baselines that route one cell at a time inevitably face a long horizon that weakens credit assignment. By contrast, PPO issues segment-level KiCAD-API actions, so its decision horizon is tied to routed segments and does not grow with grid resolution. The full grid-size sweep numbers are tabulated in Appendix L.

5.3 Comparing Routing Agents in PCBWORLD

We compare all methods in Table 3 using CP as the primary metric, with Pot., Rout., and Time reported alongside. Figure 7 shows qualitative results produced by each method.

Trained from scratch on synthetic D2-train, a compact PPO Transformer outperforms all LLM agents and the remaining rule-based routers, with only Freerouting, the strongest baseline, remaining competitive. On in-distribution D2-test, PPO matches Freerouting’s perfect CP and Rout. (both 1.00) and leads on Pot. (16.24 vs. 15.47) and Time (0.43s vs. 2.69s). On out-of-distribution D3-A, it still attains higher CP (0.86 vs. 0.80), indicating generalization. On the larger D3-B split, Freerouting outperforms PPO on both CP (0.78 vs. 0.45) and Pot. (61.06 vs. 50.20). This generalization thus reaches its limit on boards far beyond the D2-train size range (Table 2).

LLM agents leverage the engine’s native operations without any routing-specific training, and their routing ability scales with model capacity. GPT-5.4 reaches a CP of 0.96 on D2, ahead of its mini (0.58)

and nano (0.55) variants. All LLM agents nonetheless obtain 0.00 CP on D3-B, leaving large-board routing in PCBWORLD-BENCH an open challenge for current LLMs. The LLM agents are also far slower than the routers and PPO (94.04s vs. 0.43s on D2 for GPT-5.4), as every action requires a full model inference. The strong performance of a compact PPO policy suggests RL fine-tuning of LLMs as a direction toward expert routing agents.

5.4 Impact of Engine Engagement in LLM Agents

Table 4 compares three engine-engagement levels of the LLM agent on D3-A, *interactive*, *plan-and-execute*, and *engine-free*. On our primary metrics CP and Pot., the three levels rank in this order consistently across all GPT models.

Leveraging the native API is essential for design-rule-clean routing. The engine-free agent records far worse DRV than the interactive and plan-and-execute agents, since generating raw wire segments while keeping every track within the design rules is beyond current models. Smaller models degrade further. Under engine-free generation, for 10% of D3-A boards none of GPT-5.4-nano’s five rollouts even loads as a valid KiCAD board (Parse-fail, Table 4).

Although the plan-and-execute and interactive agents share the same PCBWORLD action space and both form connections, only the interactive agent keeps them design-rule-clean, so DRV and CP diverge sharply on D3-A (Table 4). As in an engineer’s routing workflow, both acting through the native API and accurate observation of the board state are essential for clean PCB routing. Figure 8 extends the D3-A comparison of Table 4 to D2. The interactive > plan-and-execute > engine-free ordering on CP and Pot. holds consistently across all GPT models on D2 as well.

5.5 Impact of Reward and Action Design in RL

Finer credit assignment improves the learned policy. We compare PPO (per-step) against the terminal-reward variants PPO (terminal)

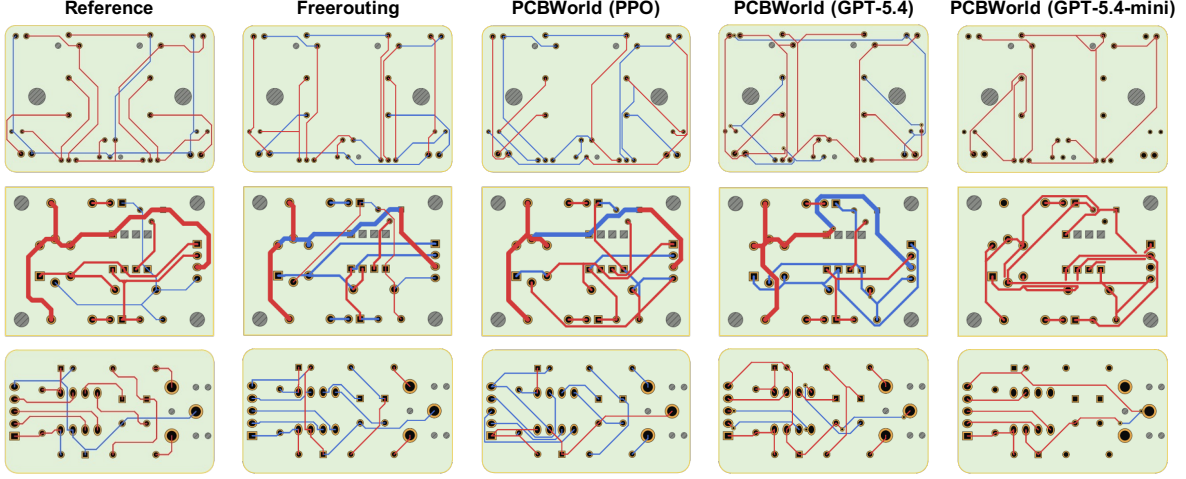
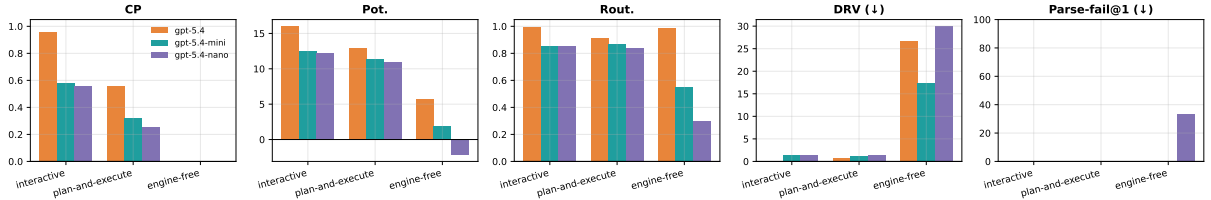
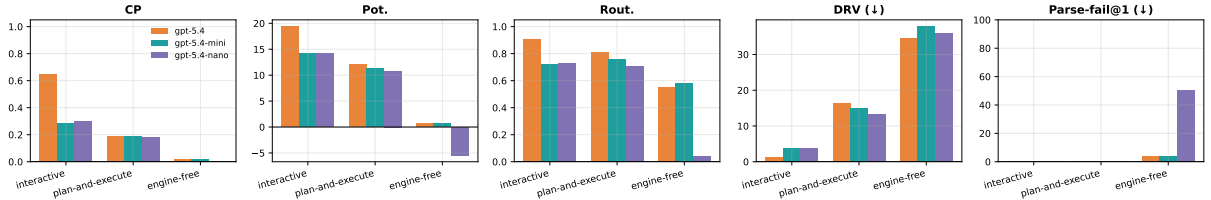


Figure 7: Routed boards on three D3-A instances. Each column shows one method’s output for the same boards. Red and blue traces mark wire segments on the two copper layers. Additional boards are shown in Figure 27 (Appendix N).



(a) D2 (Synthetic Gridless Board).



(b) D3-A (Open-source Board).

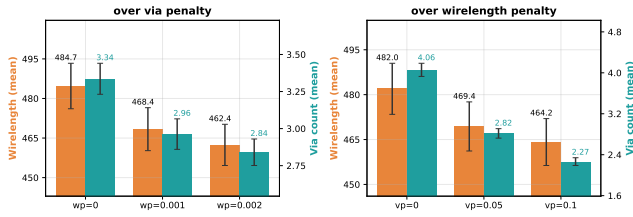
Figure 8: Engine engagement in the LLM agent on D2 and D3-A across interactive, plan-and-execute, and engine-free modes. Panels report CP, Pot., Rout., and DRV under the @5 protocol and Parse-fail@1, the fraction of rollouts that fail to load, plotted in percent; Table 4 reports Parse-fail under the default @5 protocol.

and GRPO (Table 3). All three train to competent policies, with PPO (per-step) the strongest overall, showing the highest Pot. on all three splits and CP that matches or exceeds the other two. PPO (terminal) and GRPO both learn from the completed board alone. They perform comparably on D2 and D3-A, while PPO (terminal) holds up better on the larger D3-B boards (CP 0.38 vs. 0.10, Pot. 44.33 vs. 30.00). Since PPO (terminal) differs from PPO only in reward density, the gain traces to the denser per-step shaping signal (§3.2).

The policy does not rely on engine auto-completion. We trained PPO (w/o finish), removing the strongest form of engine assistance from its action space (Table 1), to test whether the agent can lay every segment on its own. Routing quality remains comparable on all three splits, with CP on D3-A improving to 0.94 (Table 3). The geometrically feasible routing thus comes from the learned policy rather than the engine’s auto-completion. We keep finish in the default action space because it is a native operation of the KiCAD engine, which PCBWORLD exposes as is.

Table 4: Engine engagement in the LLM agent on D3-A. We compare the interactive, plan-and-execute, and engine-free modes.

Model	Mode	CP $\uparrow$	Pot. $\uparrow$	Rout. $\uparrow$	DRV $\downarrow$	Parse-fail $\downarrow$
Reference	–	1.00	23.16	1.00	0.00	–
GPT-5.4	interactive	0.65	19.42	0.91	1.31	0.00
	plan-and-execute	0.19	12.14	0.81	16.28	0.00
	engine-free	0.02	0.76	0.55	34.58	0.00
GPT-5.4-mini	interactive	0.28	14.22	0.72	3.88	0.00
	plan-and-execute	0.19	11.32	0.76	14.85	0.00
	engine-free	0.02	0.70	0.58	37.80	0.00
GPT-5.4-nano	interactive	0.30	14.16	0.73	3.84	0.00
	plan-and-execute	0.18	10.80	0.71	13.26	0.00
	engine-free	0.00	-5.45	0.04	35.89	0.10

**Figure 9: Reward weight sweep (3 $\times$ 3 factorial). Bars show marginal means with error bars. Wirelength: left axis (orange); via count: right axis (teal).**

Controllability through penalty weights. We test whether the policy responds to fine-grained routing objectives by sweeping the wirelength weight $\lambda_w \in \{0, 0.001, 0.002\}$ and the via weight $\lambda_v \in \{0, 0.05, 0.1\}$ in a 3 $\times$ 3 factorial design (Figure 9). We analyze the full factorial results by aggregating them along each axis. Averaged over all via-weight settings, increasing λ_w consistently reduces wirelength from 484.8 mm to 468.4 mm and 462.4 mm. Likewise, averaged over all wirelength-weight settings, increasing λ_v reduces the via count from 4.06 to 2.82 and 2.27. Although wirelength and via count are coupled in PCB routing, these marginal trends show that each penalty acts as a consistent optimization signal for its associated cost. Overall, the policy adapts its routing behavior to the specified objective rather than collapsing to a single operating point.

6 Related Work

Rule-based PCB routers. Automatic PCB routing has a long algorithmic history, spanning grid-based maze search since Lee’s algorithm [26] and gridless techniques [50], with rip-up and reroute as the central iterative heuristic [11, 34]. Academic refinements continue this line [8, 16, 33], and rule-based routers remain central in practice, from the open-source routers evaluated here [3, 12, 15] to commercial EDA suites [1, 6]. Since their prior knowledge is encoded in manually designed rules and heuristics, performance depends on objective- and board-specific tuning.

RL for PCB routing. Existing RL work narrows the agent’s action interface in one of two ways. The first family casts routing as

cell-by-cell movement on a discretized grid [5, 18, 31, 38]; at the grid resolution production boards require, the search space grows impractically large. The second family routes only indirectly: the agent selects high-level options for a fixed auto-router, net order [30, 32, 61, 62], routing pattern [7], or fanout locations [27], while the router draws all geometry, capping quality at the router’s ceiling. In neither family does the agent draw board geometry through the engine’s native operations. PCBWORLD instead exposes the engine’s native routing operations as the agent’s action space [24, 25], and derives the learning signal from the engine’s design-rule check rather than from a hand-crafted geometric proxy.

LLM agents for EDA. LLM agents that pair reasoning with tool calls [60] are beginning to reach EDA. Within PCB, PCB-Bench [28] evaluates an LLM’s board interpretation but not its routing ability. Other work either proceeds in a single open-loop pass or uses the engine as a checker. The former generates EDA scripts or hardware description language (HDL) code [36, 37, 49, 54]. The latter revises the HDL on compiler and simulation errors [4, 19, 51, 57] or a schematic on electrical-rule violations [63]. In PCBWORLD, the agent acts through the engine’s native operations in a closed loop, and the engine checks every intermediate board rather than only the completed artifact.

Agent benchmarks beyond EDA. Outside EDA, LLM-based design generation has been studied most actively in 3D CAD. These methods generate CAD operation sequences or parametric code from text and images [14, 29, 53, 55]. Benchmarks score reconstruction of a labeled target 3D shape, a softer standard than EDA’s design rules [52]. Agent benchmarks in general software domains score code patches and tool-use episodes, with little geometry at stake [22, 43, 45, 58, 59]. Computer-use environments and GUI agents operate software through the screen as a human does [9, 20, 56]. If applied to an EDA tool, they would come closest to an engineer’s workflow. Acting through the screen costs them threefold: (i) pixels only approximate the exact board geometry, turning routing into a POMDP; (ii) precise cursor control remains unreliable for current agents; (iii) rendering every step slows both training and evaluation. PCBWORLD bypasses the screen altogether, operating on the exact board state.

7 Conclusion

We present PCBWORLD, a Gym environment that wraps the open-source KiCAD EDA engine and formulates PCB routing as an MDP without surrogate abstraction. Two wrappers let RL and LLM agents train and act in the same environment. Built on PCBWORLD, PCBWORLD-BENCH pairs two synthetic board generators (D1, D2) with 679 curated open-source boards reserved for zero-shot evaluation (D3), benchmarking geometric feasibility reasoning under a method-agnostic evaluation protocol. Empirically, the KiCAD-API action interface outperforms both grid-action RL and engine-free LLM generation. A compact PPO policy trained on synthetic boards transfers zero-shot to open-source boards, approaching a rule-based router refined over decades. We open-source PCBWORLD and PCBWORLD-BENCH as an engine-grounded foundation for learning-based EDA research.

References

- [1] Altium. 2026. Altium Designer: The Industry's Leading PCB Design Software. <https://www.altium.com/altium-designer>
- [2] Akshay Badagabettu, Sai Sravan Yarlagadda, and Amir Barati Farimani. 2024. Query2CAD: Generating CAD models using natural language queries. *arXiv preprint arXiv:2406.00144*. doi:10.48550/arXiv.2406.00144
- [3] Brian Benchoff. 2025. OrthoRoute: A GPU-accelerated PCB autorouter for KiCad. <https://github.com/bbenchoff/OrthoRoute>
- [4] Jason Blocklove, Shailja Thakur, Benjamin Tan, Hammond Pearce, Siddharth Garg, and Ramesh Karri. 2025. Automatically Improving LLM-based Verilog Generation using EDA Tool Feedback. *ACM Transactions on Design Automation of Electronic Systems* 30, 6 (2025), 1–26. doi:10.1145/3723876
- [5] Clément Bonnet, Daniel Luo, Donal John Byrne, Shikha Surana, Sasha Abramowitz, Paul Duckworth, Vincent Coyette, Laurence Illing Midgley, Elshadai Tegegn, Tristan Kalloniatis, Omayma Mahjoub, Matthew Macfarlane, Andries Petrus Smit, Nathan Grinsztajn, Raphael Boige, Cemlyn Neil Waters, Mohamed Ali Mimouni, Ulrich Armel Mbou Sob, Ruan John de Kock, Siddarth Singh, Daniel Furelos-Blanco, Victor Le, Arnau Pretorius, and Alexandre Laterre. 2024. Jumanji: A Diverse Suite of Scalable Reinforcement Learning Environments in JAX. In *International Conference on Learning Representations (ICLR)*.
- [6] Cadence Design Systems. 2026. Allegro X Design Platform: PCB and System Design. https://www.cadence.com/en_US/home/tools/pcb-design-and-analysis/allegro-x-design-platform.html
- [7] Hao Chen, Kai-Chieh Hsu, Walker J. Turner, Po-Hsuan Wei, Keren Zhu, David Z. Pan, and Haoxing Ren. 2023. Reinforcement Learning Guided Detailed Routing for Custom Circuits. In *Proceedings of the 2023 International Symposium on Physical Design (ISPD)*. 26–34. doi:10.1145/3569052.3571874
- [8] Jiarui Chen, Yujing Zhou, Qinghai Liu, and Xinhong Zhang. 2023. A Novel Global Routing Algorithm for Printed Circuit Boards Based on Triangular Grid. *Electronics* 12, 24 (2023), 4942. doi:10.3390/electronics12244942
- [9] Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Yantao Li, Jianbing Zhang, and Zhiyong Wu. 2024. SeeClick: Harnessing GUI Grounding for Advanced Visual GUI Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Bangkok, Thailand, 9313–9332. doi:10.18653/v1/2024.acl-long.505
- [10] Clyde F. Coombs, Jr. and Happy T. Holden (Eds.). 2016. *Printed Circuits Handbook* (7 ed.). McGraw Hill. <https://www.mheducation.com/highered/mhp/product/printed-circuits-handbook-seventh-edition.html>
- [11] W. A. Dees, Jr. and P. G. Karger. 1982. Automated rip-up and reroute techniques. In *Proceedings of the 19th Design Automation Conference (DAC)*. 432–439. doi:10.1145/800263.809241
- [12] Freerouting Project. 2026. Freerouting: Open-source PCB Autorouter. <https://github.com/freerouting/freerouting>
- [13] Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. 2021. Datasheets for datasets. *Commun. ACM* 64, 12 (2021), 86–92. doi:10.1145/3458723
- [14] Yandong Guan, Xilin Wang, XiMing Xing, Jing Zhang, Dong Xu, and Qian Yu. 2025. CAD-Coder: Text-to-CAD Generation with Chain-of-Thought and Geometric Reward. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 38. 59765–59789.
- [15] Andy Haas. 2026. KiCad Routing Tools. <https://github.com/drandyhaas/KiCadRoutingTools>
- [16] Youbiao He. 2024. *Towards Automated PCB Routing: Leveraging Machine Learning and Heuristic Techniques*. Ph. D. Dissertation. Iowa State University. doi:10.31274/td-20240617-74
- [17] Youbiao He, Jacob Frieden, Hebi Li, Roba Abbajabal, Ge Luo, and Forrest Sheng Bao. 2024. PCBench: A Dataset for Printed Circuit Board Routing. <https://github.com/PCBench/PCBench> DAC 2024 work-in-progress poster.
- [18] Youbiao He, Hebi Li, Jin Tian, and Forrest Sheng Bao. 2022. Circuit Routing Using Monte Carlo Tree Search and Deep Reinforcement Learning. In *2022 International Symposium on VLSI Design, Automation and Test (VLSI-DAT)*. 1–5. doi:10.1109/VLSI-DAT54769.2022.9768074
- [19] Chia-Tung Ho, Haoxing Ren, and Bruce Khailany. 2025. VerilogCoder: Autonomous Verilog Coding Agents with Graph-based Planning and Abstract Syntax Tree (AST)-based Waveform Tracing Tool. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, Vol. 39. 300–307. doi:10.1609/aaai.v39i1.32007
- [20] Wenyi Hong, Wei Han Wang, Qingsong Lv, Jiazhen Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxiao Dong, Ming Ding, and Jie Tang. 2024. CogAgent: A Visual Language Model for GUI Agents. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 14281–14290. doi:10.1109/CVPR52733.2024.01354
- [21] IPC. 2003. IPC-2221A-2003: Generic Standard on Printed Board Design. <https://webstore.ansi.org/standards/ipc/ipc2221a2003> Association Connecting Electronics Industries.
- [22] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world Github Issues?. In *International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=VTF8yNQm66>
- [23] R. S. Khandpur. 2006. *Printed Circuit Boards: Design, Fabrication, Assembly and Testing*. McGraw-Hill. <https://search.worldcat.org/title/62032512>
- [24] KiCad Project. 2026. KiCad API Python Bindings. <https://docs.kicad.org/kicad-python-main/>
- [25] KiCad Project. 2026. KiCad EDA Suite. <https://www.kicad.org>
- [26] C. Y. Lee. 1961. An Algorithm for Path Connections and Its Applications. *IRE Transactions on Electronic Computers* EC-10, 3 (1961), 346–365. doi:10.1109/TEC.1961.5219222
- [27] Haiyun Li, Jixin Zhang, Ning Xu, and Mingyu Liu. 2023. FanoutNet: A Neuralized PCB Fanout Automation Method Using Deep Reinforcement Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, Vol. 37. 8554–8561. doi:10.1609/aaai.v37i7.26030
- [28] Jindong Li, Lianrong Chen, Bin Yang, Jiadong Zhu, Ying Wang, Yuzhe Ma, and Menglin Yang. 2026. PCB-Bench: Benchmarking LLMs for Printed Circuit Board Placement and Routing. In *The Fourteenth International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=Q5QLu7XTWx>
- [29] Xingang Li, Yuewan Sun, and Zhenghui Sha. 2025. LLM4CAD: Multimodal Large Language Models for Three-Dimensional Computer-Aided Design Generation. *Journal of Computing and Information Science in Engineering* 25, 2 (2025), 021005. doi:10.1115/1.4067085
- [30] Haiguang Liao, Qingyi Dong, Xuliang Dong, Wentai Zhang, Wangyang Zhang, Weiwei Qi, Elias Fallon, and Levent Burak Kara. 2020. Attention Routing: Track-Assignment Detailed Routing Using Attention-Based Reinforcement Learning. In *Proceedings of the ASME International Design Engineering Technical Conferences and Computers and Information in Engineering Conference (IDETC-CIE)*. V11AT11A002. doi:10.1115/DETC2020-22219
- [31] Haiguang Liao, Wentai Zhang, Xuliang Dong, Barnabás Póczos, Kenji Shimada, and Levent Burak Kara. 2020. A Deep Reinforcement Learning Approach for Global Routing. *Journal of Mechanical Design* 142, 6 (2020), 061701. doi:10.1115/1.4045044
- [32] Yin-Chi Liao, Sheng-Xin Pan, and Po-Jui Chiang. 2026. Automation of PCB Autorouting via World-Model Reinforcement Learning and Freerouting Integration. *Expert Systems with Applications* 311 (2026), 131424. doi:10.1016/j.eswa.2026.131424
- [33] Ting-Chou Lin, Devon Merrill, Yen-Yi Wu, Chester Holtz, and Chung-Kuan Cheng. 2021. A Unified Printed Circuit Board Routing Algorithm With Complicated Constraints and Differential Pairs. In *Proceedings of the 26th Asia and South Pacific Design Automation Conference (ASP-DAC)*. 170–175. doi:10.1145/3394885.3431568
- [34] Ralph Linsker. 1984. An iterative-improvement penalty-function-driven wire routing system. *IBM Journal of Research and Development* 28, 5 (1984), 613–624. doi:10.1147/RD.285.0613
- [35] Linux Foundation. 2019. KiCad Joins Linux Foundation to Advance Electronic Design Automation. <https://www.linuxfoundation.org/press/press-release/kicad-joins-linux-foundation-to-advance-electronic-design-automation> Press release.
- [36] Mingjie Liu, Teodor-Dumitru Ene, Robert Kirby, Chris Cheng, Nathaniel Pinckney, Rongjian Liang, Jonah Alben, Himyanshu Anand, Sanmitra Banerjee, Ismet Bayraktaroglu, Bonita Bhaskaran, Bryan Catanzaro, Arjun Chaudhuri, Sharon Clay, Bill Dally, Laura Dang, Parikshit Deshpande, Siddhanth Dhodhi, Sameer Halepote, Eric Hill, Jia Shang Hu, Sumit Jain, Ankit Jindal, Bruce Khailany, George Kokai, Kishor Kunal, Xiaowei Li, Charley Lind, Hao Liu, Stuart Oberman, Sujeet Omar, Ghasem Pasandi, Sreedhar Pratty, Jonathan Raiman, Ambar Sarkar, Zhengjiang Shao, Hanfei Sun, Pratik P. Suthar, Varun Tej, Walker Turner, Kaizhe Xu, and Haoxing Ren. 2023. ChipNeMo: Domain-Adapted LLMs for Chip Design. *arXiv preprint arXiv:2311.00176*. doi:10.48550/arXiv.2311.00176
- [37] Mingjie Liu, Nathaniel Pinckney, Bruce Khailany, and Haoxing Ren. 2023. Invited Paper: VerilogEval: Evaluating Large Language Models for Verilog Code Generation. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. 1–8. doi:10.1109/ICCAD57390.2023.10323812
- [38] Omayma Mahjoub, Sasha Abramowitz, Ruan John De Kock, Wiem Khelifi, Simon Verster Du Toit, Jemma Daniel, Louay Ben Nessim, Louise Beyers, Juan Claude Formanek, Liam Clark, and Arnau Pretorius. 2025. Sable: A Performant, Efficient and Scalable Sequence Model for MARL. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*. 42579–42614. <https://proceedings.mlr.press/v267/mahjoub25a.html>
- [39] Larry McMurchie and Carl Ebeling. 1995. PathFinder: a negotiation-based performance-driven router for FPGAs. In *Proceedings of the 1995 ACM Third International Symposium on Field-Programmable Gate Arrays (FPGA95)*. ACM, 111–117. doi:10.1145/201310.201328
- [40] Andrew Y. Ng, Daishi Harada, and Stuart Russell. 1999. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning (ICML)*. 278–287. <https://dl.acm.org/doi/10.5555/645528.657613>
- [41] OpenAI. 2026. Introducing GPT-5.4. <https://openai.com/index/introducing-gpt-5-4/>

- [42] OpenAI. 2026. Introducing GPT-5.4 mini and nano. <https://openai.com/index/introducing-gpt-5-4-mini-and-nano/>
- [43] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. 2025. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*. 48371–48392. <https://proceedings.mlr.press/v267/patil25a.html>
- [44] Dacheng Qi, Chenyu Wang, Jingwei Xu, Tianzhe Chu, Zibo Zhao, Wen Liu, Wenrui Ding, Yi Ma, and Shenghua Gao. 2026. Pointer-CAD: Unifying B-Rep and Command Sequences via Pointer-based Edges & Faces Selection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 17377–17387. https://openaccess.thecvf.com/content/CVPR2026/html/Qi_Pointer-CAD_Unifying_B-Rep_and_Command_Sequences_via_Pointer-based_Edges_CVPR_2026_paper.html
- [45] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=dHng2O0Jjr>
- [46] Qwen Team. 2026. Qwen3.5: Towards Native Multimodal Agents. <https://qwen.ai/blog?id=qwen3.5>
- [47] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. arXiv preprint arXiv:1707.06347. doi:10.48550/arXiv.1707.06347
- [48] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv preprint arXiv:2402.03300. doi:10.48550/arXiv.2402.03300
- [49] Utsav Sharma, Bing-Yue Wu, Sai Rahul Dhanvi Kankipati, Vidya A. Chhabria, and Austin Rovinski. 2024. OpenROAD-Assistant: An Open-Source Large Language Model for Physical Design Tasks. In *Proceedings of the 2024 ACM/IEEE International Symposium on Machine Learning for CAD (MLCAD)*. 1–7. doi:10.1145/3670474.3685960
- [50] Naveed A. Sherwani. 1999. *Algorithms for VLSI Physical Design Automation* (3rd ed.). Kluwer Academic Publishers. doi:10.1007/b116436
- [51] Yunda Tsai, Mingjie Liu, and Haoxing Ren. 2024. RTLFixer: Automatically Fixing RTL Syntax Errors with Large Language Model. In *Proceedings of the 61st ACM/IEEE Design Automation Conference (DAC)*. 1–6. doi:10.1145/3649329.3657353
- [52] Liang Wang, Heng Meng, Zekai Xiang, Jin Liu, Pingyi Zhou, Litao Chen, and Yongqiang Tang. 2026. Text2CAD-Bench: A Benchmark for LLM-based Text-to-Parametric CAD Generation. arXiv preprint arXiv:2605.18430. doi:10.48550/arXiv.2605.18430
- [53] Siyu Wang, Cailian Chen, Xinyi Le, Qimin Xu, Lei Xu, Yanzhou Zhang, and Jie Yang. 2025. CAD-GPT: Synthesising CAD Construction Sequence with Spatial Reasoning-Enhanced Multimodal LLMs. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, Vol. 39. 7880–7888. doi:10.1609/aaai.v39i8.32849
- [54] Haoyuan Wu, Zhuolun He, Xinyun Zhang, Xufeng Yao, Su Zheng, Haisheng Zheng, and Bei Yu. 2024. ChatEDA: A Large Language Model Powered Autonomous Agent for EDA. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 43, 10 (2024), 3184–3197. doi:10.1109/TCAD.2024.3383347
- [55] Rundui Wu, Chang Xiao, and Changxi Zheng. 2021. DeepCAD: A Deep Generative Network for Computer-Aided Design Models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 6752–6762. doi:10.1109/ICCV48922.2021.00670
- [56] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. 2024. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 37. 52040–52094. doi:10.52202/079017-1650
- [57] Ke Xu, Jialin Sun, Yuchen Hu, Xinwei Fang, Weiwei Shan, Xi Wang, and Zhe Jiang. 2024. MEIC: Re-thinking RTL Debug Automation Using LLMs. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. 1–9. doi:10.1145/3676536.3676801
- [58] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 37. 50528–50652. doi:10.52202/079017-1601
- [59] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2025. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. In *The Thirteenth International Conference on Learning Representations (ICLR)*.
- [60] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=WE_vluYUL-X
- [61] Zhanwen Zhou, Hankz Hankui Zhuo, Xiaowu Zhang, and Qiyuan Deng. 2023. XRoute Environment: A Novel Reinforcement Learning Environment for Routing. arXiv preprint arXiv:2305.13823. doi:10.48550/arXiv.2305.13823
- [62] Zhanwen Zhou, Hankz Hankui Zhuo, Jinghua Zhou, and Wushao Wen. 2025. Transformer-based Reinforcement Learning for Net Ordering in Detailed Routing. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence (IJCAI)*. 9492–9500. doi:10.24963/ijcai.2025/1055
- [63] Huanghaohe Zou, Peng Han, Emad Nazerian, Mafu Zhang, Zhicheng Guo, and Alex Q. Huang. 2026. PCBSchemaGen: Reward-Guided LLM Code Synthesis for Printed Circuit Boards (PCB) Schematic Design with Structured Verification. arXiv preprint arXiv:2602.00510v2. doi:10.48550/arXiv.2602.00510

A Notation Glossary

Table 5: Notation.

Symbol	Description
$n_{\text{drv}}(\cdot)$	Number of design rule violations (DRVs)
$n_{\text{via}}(\cdot)$	Number of vias
$\ell(\cdot)$	Total wirelength
λ_w, λ_v	Wirelength and via weights in Equation (1)
$f_{\text{drv}}(\cdot)$	Concave DRV penalty in the potential Φ , computed from the per-net violation decomposition of s (Equation (3))
$\mathcal{R}$	Set of board states
s	Board state, represented as a nested dictionary
s_t	Board state at step t ; $t = T$ is the terminal step
$\Phi(\cdot)$	Potential function
r_T	Terminal reward, $\Phi(s_T) - \Phi(s_0)$ (the MDP reward)
r_t	Per-step reward, $\Phi(s_{t+1}) - \Phi(s_t)$ (shaping form)
G	Grid size

B System Architecture (PCBWORLD)

This appendix distinguishes carefully among the routing method’s decisions, Gym-env mediation, the wrapper that surfaces the 58 step-level APIs, and the C++ binding underneath. Confusing these four layers is the most common source of misreading the state, action, and reward decomposition in Appendix C, so we fix the layered picture here. Figure 10 gives the agent–environment view of the stack and the RL signals connecting the two sides.

B.1 Layering the stack

At the top, three classes of routing methods share an identical observation and action interface: a deterministic scripted rule-based baseline, an RL agent built on a from-scratch Transformer, and an LLM agent that issues tool calls. They all enter the stack through KiCadHLEnv at L4, the Gymnasium environment facade that owns four sub-modules: an action dispatcher for command routing, an observation builder, a reward potential composer, and an action mask. The dispatcher is the only L4 sub-module that mutates state; the other three are read paths. L4 holds a single KiCadEngine instance at L3, the Python wrapper around the C++ routing engine that exposes the 58 step-level APIs the rest of the appendices catalog. L3 is the only Python call site that reaches the Python $\leftrightarrow$ C++ binding at L2, behind which sits the unmodified KiCAD C++ engine at L1.

B.2 Building observations and rewards

This subsection and the next together specify the L3 $\leftrightarrow$ L4 signal exchange that synthesizes an MDP interface from the unmodified engine, which is what makes the stack an RL environment rather than a Python SDK over the 58 APIs. Every read crossing back from L3 into Python is packaged into one of three dataclasses. BoardSnapshot is the geometry bundle that the observation builder consumes. RewardSnapshot is the unrouted, wirelength, and DRC bundle that the reward potential composer folds into $\Phi(s)$. RoutingSessionState is the routing FSM bundle that lets the action mask and the action dispatcher decide which actions are admissible. Because these three are the only read channels, the L3 $\leftrightarrow$ L4 coupling reduces to which engine method populates which dataclass field, which is exactly the reading the dual-tag analysis in Appendix C.2 formalizes.

B.3 Dispatching actions

The three method classes do not call the 58 step-level APIs directly. They emit a single high-level action through the action dispatcher at L4, which translates that action into a small subset of L3 mutators. Appendix C catalogs the 58 APIs and their MDP components, and the high-level action surface exposed to the methods is documented in the main text under §3.2. The data contract above fixes the state, reward, and admissibility channels that a method reads, while this high-level action surface fixes the action space it emits, completing the MDP interface that L4 exposes upward.

C Mapping the API to the MDP

The agent reaches KiCAD’s PNS router through a thin Python wrapper whose public surface exposes 58 step-level methods, partitioned by the wrapper source into 14 source-code groups and mapped to 25 Action, 25 State, 3 Reward, and 5 Utility components of the MDP. These role totals reconcile with the main-text partition of §3.1: the 25 State and 3 Reward methods are the 28 DRC and board-state wrappers, and

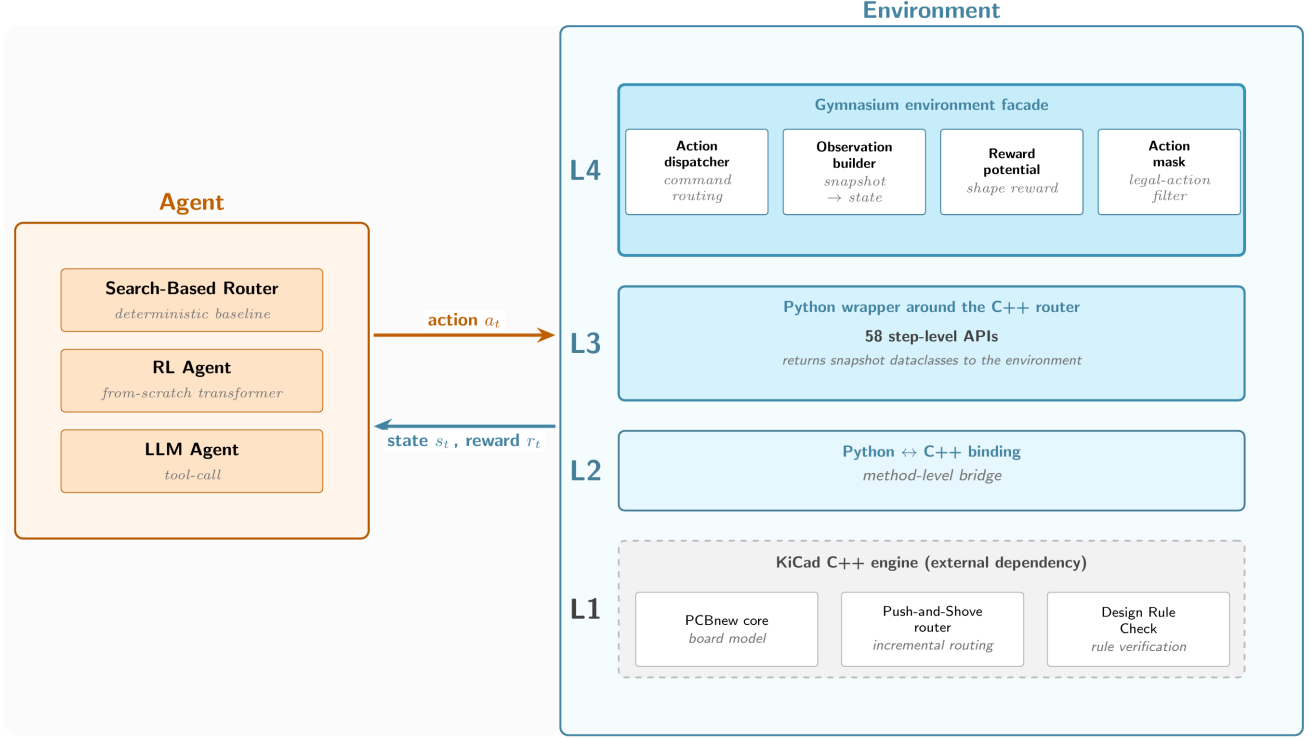


Figure 10: PCBWORLD architecture. A reinforcement-learning view of the proposed environment. Three classes of routing methods, a scripted rule-based baseline, an RL agent built on a from-scratch Transformer, and an LLM agent that issues tool calls, share a single Gymnasium environment through the standard action a_t and the state s_t , reward r_t signals. Inside the environment, our contribution spans L4 (the Gymnasium environment facade with four sub-modules: action dispatcher, observation builder, reward potential, and action mask), L3 (the Python wrapper exposing 58 step-level APIs), and L2 (the Python $\leftrightarrow$ C++ binding); layer-box heights encode relative scope, while sub-boxes detail the internal components. L1 is KiCAD’s unmodified C++ engine, namely PCBnew core, Push-and-Shove router, and Design Rule Check, drawn with a dashed border as an external dependency.

the 25 Action and 5 Utility methods are the 14 core routing APIs together with the 16 auxiliary utilities. Table 6 catalogs every method by MDP component and source-code group, and is the central artifact of this appendix. Four wrapper-only helpers are excluded from the count: close is episode teardown, get_native returns a raw C++ handle for renderers, and get_drc_result, get_board_snapshot are Python-side dataclass aggregators whose underlying getters surface individually below. get_reward_snapshot remains counted because it performs its own engine pass rather than re-bundling listed getters, as detailed in Appendix C.2. Appendix B situates this wrapper as the L3 layer of the overall PCBWORLD stack.

For a step-level method m at L3, component membership is fixed by which L4 call-sites reach m , not by what the method returns. The two subsections below organize the 58 APIs of Table 6: the Routing MDP collects the per-step State, Reward, Action, and transition components, while the miscellaneous group covers episode-level Utility methods and seven dual-tagged overlaps.

C.1 Routing MDP

State. Methods that feed the observation builder, either directly or through the BoardSnapshot aggregator. The aggregator bundles seven read-only getters spanning board geometry, ratsnest, and metadata into a single dataclass, but is excluded from the 58-method inventory because its underlying getters appear individually. The Board meta, Elements, Session, Design rules, and Snapshots groups are State.

Reward. Methods that populate fields of the RewardSnapshot dataclass that the potential composer folds into $\Phi(s)$ in §3.2. The Diagnostics group exhausts this tag, with the get_reward_snapshot aggregator joining the two raw DRC counters.

Group	<i>n</i>	Methods
Action: <i>drive PCB routing through copper placement, parameter changes, and rework (25 methods).</i>		
Configuration	6	set_routing_mode(), set_corner_mode(), set_track_width(), set_via_diameter(), set_via_drill(), reset_via_mode()
Routing	9	start_route(), move(), fix_route(), cancel_route(), finish(), undo_last_segment(), flip_posture(), toggle_via(), switch_layer()
Dragging	3	start_drag(), fix_drag(), cancel_drag()
Rework	4	delete_track_by_index(), delete_track_near(), delete_via_by_index(), delete_via_near()
Refresh, <i>a/r</i>	1	run_drc()
Refresh, <i>a/u</i>	2	build_connectivity(), set_design_rules()
State: <i>read the current PCB layout, routing session FSM, and design rules (25 methods).</i>		
Board meta	3	get_board_bbox(), get_board_net_count(), get_copper_layer_count()
Elements	7	get_tracks(), get_vias(), get_pads(), get_points(), get_ratsnest(), get_board_outline(), get_net_names()
Elements, <i>s/r</i>	3	get_track_count(), get_via_count(), get_unrouted_count()
Session	8	get_router_state_code(), is_routing(), is_dragging(), is_placing_via(), get_current_layer(), get_route_head(), get_current_net_code(), get_routing_target()
Design rules	2	get_design_rules(), get_netclass_for_net()
Snapshots	2	get_board_meta(), get_routing_session_state()
Reward: <i>score routing quality through DRC violations and routing progress (3 methods).</i>		
Diagnostics	2	get_drc_violation_count(), get_drc_violations()
Diagnostics, <i>s/r</i>	1	get_reward_snapshot()
Utility: <i>save the routed PCB and reset engine caches between episodes (5 methods).</i>		
DRC cache	1	clear_drc_cache()
I/O	1	save()
Provenance	3	get_project_path(), was_project_loaded_from_file(), was_legacy_design_settings_loaded()

Table 6: Step-level API inventory. The 58 APIs map to 25 Action, 25 State, 3 Reward, and 5 Utility methods, each further refined into one of 14 source-code groups. Dual-component rows append the secondary tag in italic after the group name, for example *a/r*; these four rows cover the seven methods whose component overlap is forced by the underlying C++ binding. The Refresh group covers engine-cache refresh operations called at step or episode boundaries, which is why its rows are dual-tagged rather than purely Action.

Action. Methods that the per-step dispatcher invokes to mutate board or router state, covering reset cleanup, per-step configuration, and within-episode action handling. The Configuration, Routing, Dragging, and Rework groups fall here.

State transition. The transition kernel $P(s_{t+1} | s_t, a_t)$ is realized entirely by the unmodified C++ engine. An Action method submits the requested mutation, the engine applies, modifies, or rejects it according to its routing and DRC logic, and the L4 facade then harvests s_{t+1} from the resulting BoardSnapshot and RoutingSessionState while RewardSnapshot supplies r_t on the same C++ pass. Inadmissible actions are pre-filtered by the action mask reading RoutingSessionState, so the engine’s reject path is defensive rather than routine, and no Python code computes the transition itself.

C.2 Miscellaneous

Utility. Methods reached only from reset or end-of-episode bookkeeping, never from the per-step decision path. The DRC cache, I/O, and Provenance groups are pure Utility.

Dual-tagged exceptions. The component assignment above admits seven dual-tagged exceptions, all forced by the underlying C++ binding rather than by taxonomic ambiguity. The three element-count getters and get_reward_snapshot share an *s/r* tag because one C++ pass populates both a State and a Reward field. run_drc carries *a/r* because its single invocation mutates the DRC cache and also refreshes the violation list the reward consumes. build_connectivity and set_design_rules carry *a/u* because they mutate engine caches only at episode boundaries and never commit copper. With these seven exceptions, 51 of 58 methods carry a single tag, so the per-component totals of 25 Action, 26 State, 7 Reward, and 7 Utility sum to 65 and exceed the unique count of 58 by exactly the dual-tagged rows.

D Visual Demonstration for Routing APIs

This appendix complements the API table of Appendix C with a visual reference for the 22 routing-related entries. Every figure shares a single template: a *scenario panel* on the left fixes the starting board and overlays a dashed plan for the intended route, an *action sequence* in the middle prints the literal `kicad_engine.py` call sequence, and *variant panels* on the right show the resulting copper under different parameter choices. Net colors are computed deterministically from the engine’s `net_code` so each net keeps a consistent color across figures: NET1 stays red, NET2 stays blue, and NET3 is rendered as the dashed plan. Layer is color-coded with *Top* in red and *Btm* in blue, following the KiCAD GUI convention. Coordinates use the KiCAD board frame, with the origin at the top-left and the *y*-axis pointing downward, and all dimensions are given in millimeters. The visualization covers twenty-one of the twenty-two APIs in five thematic groups. The remaining one, `undo_last_segment`, only mutates the in-flight head and never modifies committed copper, so a static figure cannot show its effect. Its semantics are verified by the unit test `test_undo_does_not_commit_tracks`.

Routing primitives. A routing session is bracketed by `start_route` and one of three completion calls: `fix_route`, `cancel_route`, or `finish`. Within the session, `move` re-positions the head while only `fix_route` commits a segment to copper. Figure 11 visualizes this commit-versus-no-commit boundary across a four-step session, and Figure 12 contrasts the three completion calls on a three-pad scenario. Two further session-time controls shape the geometry of the committed trace: `flip_posture` in Figure 13 mirrors the L-shape orientation chosen for asymmetric endpoints, and the layer-transition pair `toggle_via/switch_layer` in Figure 14 either drops a stand-alone via or inserts one automatically when the active layer changes.

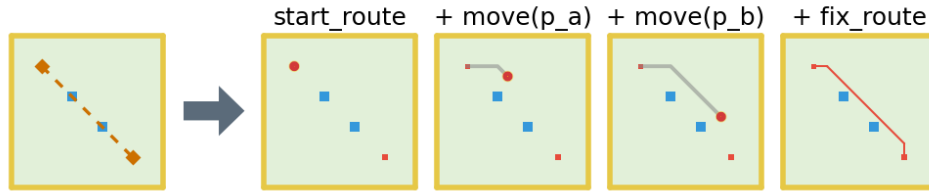


Figure 11: Virtual move versus physical commit. Within a routing session, `move` relocates the virtual head that reflects the designer’s intent and draws only a transient hint-line preview without modifying the board. `fix_route` commits the current virtual path as a physical copper object in the database and converts the preview into a permanent trace.

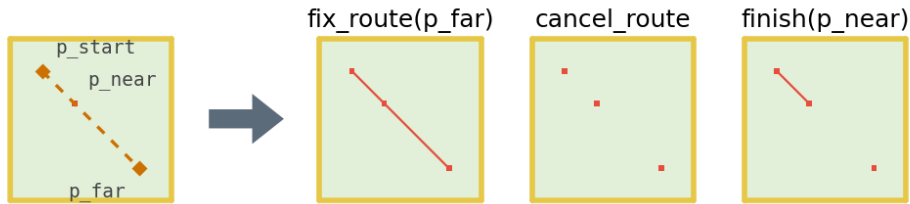


Figure 12: Session termination modes. Three distinct primitives close an open routing session. `fix_route` commits the trace at the designer-specified coordinate and ends the session. `cancel_route` discards every virtual change in the current session and restores the board to its pre-session state. `finish` invokes the PNS router to complete the route to the Euclidean-nearest unconnected pad on the current layer and ends the session.

Routing-mode controls. Two session-level switches choose how the PNS router treats existing copper and how it draws corners. Figure 15 contrasts the three strategies of `set_routing_mode`, namely `mark_as_obstacles`, `push_n_shove`, and `walkaround`, on a board pre-routed with two obstacle nets. Figure 16 contrasts the two settings of `set_corner_mode`, 45° mitering against 90° rectilinear corners, on identical diagonal endpoints. The default MITERED_45 mode is the recommended PCB practice, while the MITERED_90 mode is exposed for compatibility with legacy EDA tools but is generally avoided in modern design.

Track and via geometry. Width and via geometry are exposed as session-time setters that override the design-rule defaults for the next committed segment or via. Figure 17 sweeps `set_track_width` across three target widths on identical endpoints, the same control used in practice to differentiate power, clock, and signal nets. Figure 18 factorizes via geometry into three independent calls, `set_via_diameter`, `set_via_drill`, and `reset_via_mode`, with a final reset variant verifying that the per-call override is not sticky.

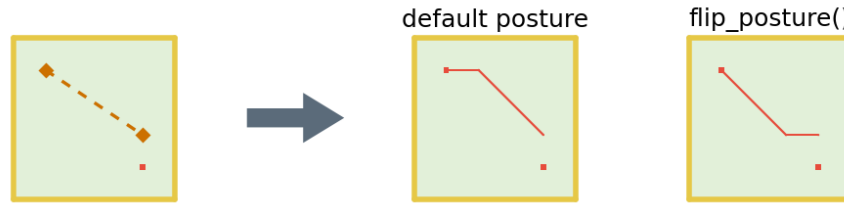


Figure 13: Corner posture control. When two endpoints are joined by an L-shape, the PNS router must choose between leaving the start pad horizontally first or vertically first. `flip_posture()` reverses this corner direction and allows the routing efficiency to be tuned dynamically within the available design space.

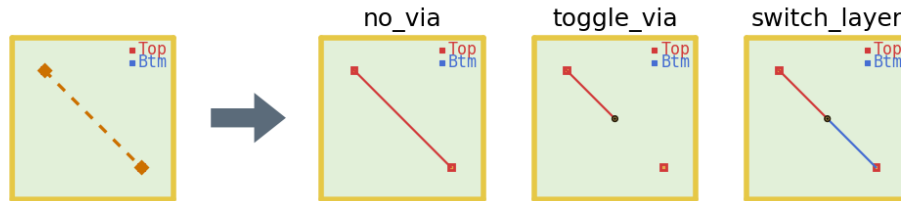


Figure 14: Via placement and layer transition. In a multilayer board, electrical connections between different layers are mediated by vias. `toggle_via` drops a vertical connection at the current location, while `switch_layer` changes the active routing layer and automatically inserts a transition via to connect the two layers.

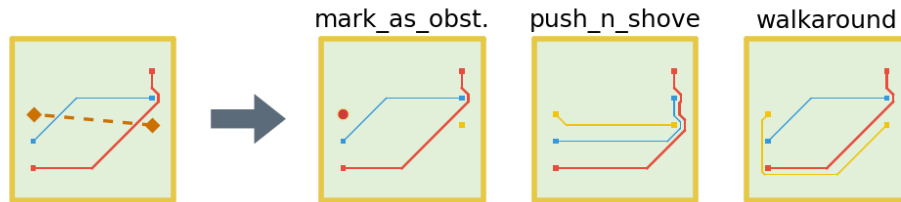


Figure 15: Obstacle interaction policies. `set_routing_mode` chooses one of three policies for interacting with existing copper. `mark_as_obstacles` performs collision detection and aborts trace generation when a clearance constraint would be violated. `push_n_shove` pushes neighboring tracks aside to dynamically clear space for the new path. `walkaround` preserves the existing copper geometry and searches for an optimal detour around it.

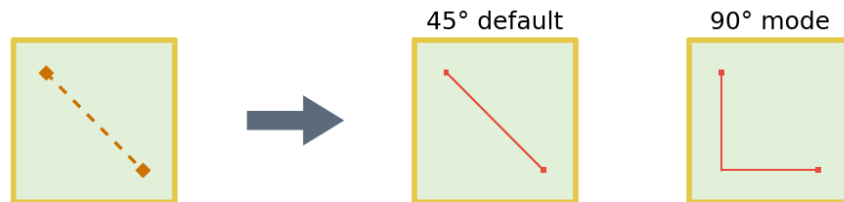


Figure 16: Trace corner geometry. `set_corner_mode` controls the angular format at trace corners. The industry-standard 45° miter, `MITERED_45`, is optimized to minimize signal loss and avoid manufacturing defects. The 90° rectilinear corner, `MITERED_90`, is reserved for cases that require special geometric alignment.

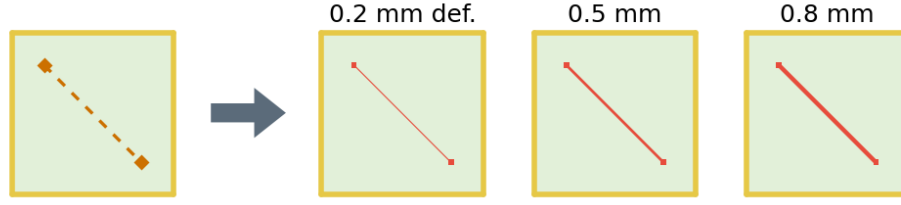


Figure 17: Trace width control. `set_track_width` explicitly sets the physical conductor width. Width is the key parameter that determines characteristic impedance and current-carrying capacity, distinguishing wide power traces that carry large currents from thin signal traces that carry fine signals.

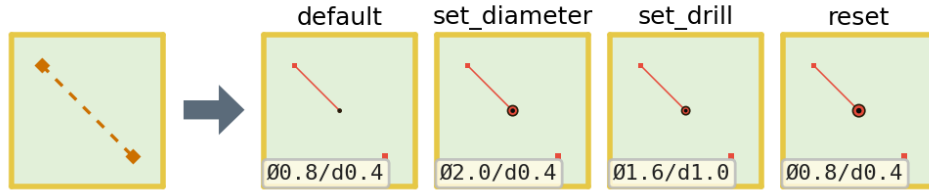


Figure 18: Via geometry and reset. The outer diameter and inner drill of a via are adjusted individually and independently of the design rules. `reset_via_mode` clears such temporary overrides and restores the system’s default design standard, preserving design consistency.

Drag-to-modify. Once a track is committed, the engine permits non-destructive geometric modification through a drag handle on its midpoint. Figure 19 shows the three primitives `start_drag`, `fix_drag`, and `cancel_drag` on a single pre-routed diagonal, contrasting the committed and aborted shapes side by side. The drag family complements the deletion primitives below by providing an in-place modification path that does not require removal followed by re-routing.

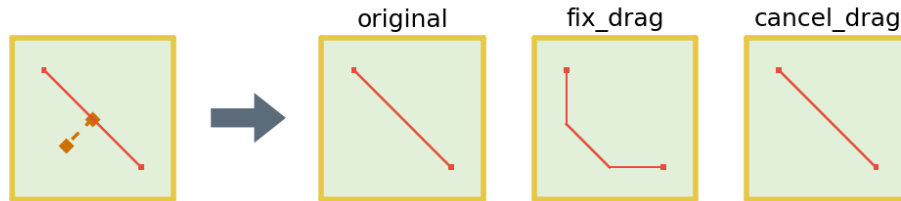


Figure 19: Trace drag-to-modify. Pulling the midpoint of an already committed trace allows its geometry to be reshaped flexibly. This is a non-destructive editing path used when local route optimization is needed without disturbing the existing connectivity.

Track and via removal. Tracks and vias can be removed either by integer position in the engine’s internal list or by spatial coordinate within a tolerance. Figure 20 contrasts `delete_track_by_index` and `delete_track_near` on a chained three-segment L, and Figure 21 contrasts `delete_via_by_index` and `delete_via_near` on a multi-via trace. The two addressing modes are functionally equivalent at the engine level but differ in agent ergonomics: index addressing is convenient for replay or rollback patterns, while coordinate addressing is convenient when the agent identifies the target by its board coordinates.

E State-dictionary schema

The environment projects the raw KiCAD `.kicad_pcb` S-expression through four progressively narrower views: the C++ *engine* (KiCadEngine, the authoritative live state), a JSON *Gym observation* (KiCadHLEnv), an *RL token stream* (BatchedStateTokenizer), and an *LLM text prompt* (S-expression / XML rendering). Table 7 lists every logical field once and marks where it appears in the chain. The mutability split of the main text’s nested dictionary (§3.2) maps onto these fields directly: **Board_static** covers the pad and outline geometry and the design-rule

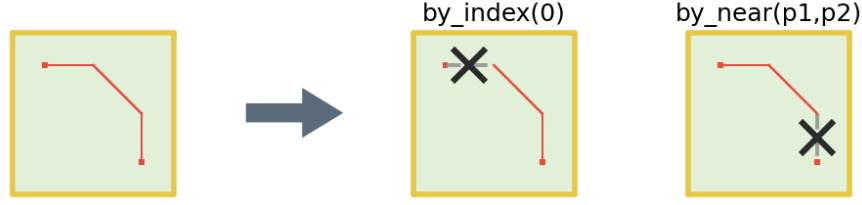


Figure 20: Track deletion methods. A track object can be identified and removed by two complementary logical approaches. `delete_track_by_index` addresses the engine’s internal list by integer position to identify the target precisely. `delete_track_near` addresses the same target by physical coordinate, providing an intuitive interface for an agent that reasons in board coordinates.

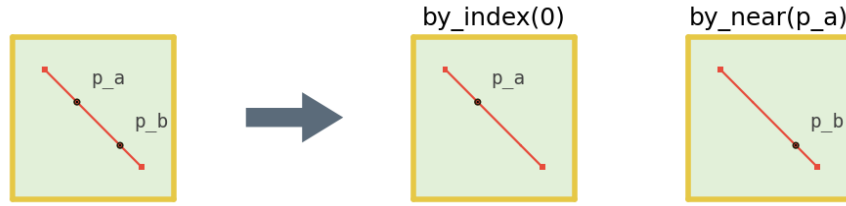


Figure 21: Via deletion methods. By the same logic as track removal, the vertical connection object is selectively removed either by engine index or by spatial coordinate.

fields, while **Routing_geometry** covers the track, via, connectivity, router-state, and DRC fields that change during routing. The chain is *largely lossless in the sense that matters for routing*. Every field the agent does not see is one the engine still retains and enforces on the agent’s behalf.

Lossless where it matters. Every row of Table 7 that drops out below the engine column does so for one of three reasons, none of which compromises the agent’s ability to plan a legal route.

(1) *The engine still enforces it.* Via drill and pad rotation disappear from the agent’s view but are used by the C++ DRC engine when it scores the agent’s track attempts; the agent decides “place a via here”, not “place a via with this drill”. Per-netclass design rules are collapsed for the agent into a single strictest envelope, but on `net_select` the environment pushes the resolved class’s track width, via diameter and drill into the PNS router, so the engine routes at the correct per-class dimensions even though the agent only sees the envelope.

(2) *The field is derivable from what the agent does see.* The `state_code` duplicates `is_routing` plus the phase flag. `is_dragging` is non-zero only in GUI drag sessions the agent never triggers. Track and via widths in the LLM prompt are recoverable from `board_constraints` for routing-quality reasoning.

(3) *The field is not relevant to routing decisions.* Pad names, footprint references, layer-stack-up dielectrics, 3D models, UUIDs and timestamps are dropped throughout; the schema synthesizes its own short, episode-local IDs ($P0$, $T0$, ...) for addressing.

The genuine losses are limited to two: copper pours ((`zone ...`)), which the engine itself does not represent in the PNS routable space, and per-class rule differentiation as visible to the agent. Both are recoverable upgrades rather than architectural barriers. The file’s information already reaches the engine, and surfacing it would extend the projection rather than replace it.

Differences between the RL and LLM views are representational, not informational. The two agent-facing layers see the same underlying state but consume it differently. The RL stream emits a discrete (`action_type`, `pointer_idx`, `routing_mode`) triple over a candidate pool synthesized from the current net’s pads, vias, track endpoints and a directional grid; coordinates are therefore selected from a finite snap-set. The LLM emits free-form text constrained by a guided-decoding regex, allowing it to invent detour waypoints anywhere within the bounding box. The RL stream additionally encodes the top-32 DRC violations and an augmentation context for coordinate symmetries; the LLM compensates with multi-turn history, a no-effect retrofit marker on `start_route+empty-effect` pairs, and a rejection-streak slot that surfaces consecutive identical mask rejections. In both cases the engine is the final filter. Mask rejection, pointer-out-of-pool (RL only) and parse failure (LLM only) all collapse to an `idle` fallback, so the PNS router never sees illegal input regardless of which consumer is driving.

Table 7: Field-by-field schema across the projection layers. ✓ = present, ○ = present in reduced form, ✗ = absent.

Field	.kicad_pcb origin	Engine	JSON	RL	LLM
<i>Geometry</i>					
Board bbox / copper layer count	derived from outline	✓	✓	✓	✓
Board outline (Edge.Cuts)	(gr_line/arc/...)	○ ^a	✓	✓	○ ^b
Pad geometry (xy, w, h, layer)	(pad ...)	✓	✓	✓	○ ^c
Pad metadata (name, ref, rotation, shape)	(pad/footprint ...)	✓	○ ^d	✗	○ ^d
Track segment	(segment ...)	✓	✓	✓	○ ^b
Via outer diameter / layer span	(via ...)	✓	✓	✓	○ ^e
Via drill diameter	(via ...)	✓	✗	✗	✗
Zone / copper pour	(zone ...)	✗	✗	✗	✗
<i>Net / connectivity</i>					
Net membership	(net N)	✓	✓	✓ ^f	✓
Unconnected / NPTH pads	net 0, np_thru_hole	✓	✓	✗	○ ^g
Ratsnest (unrouted edges)	— (computed)	✓	✓	✓	✓
<i>Design rules</i>					
Per-netclass distribution	(setup) / .kicad_pro	✓	✗	✗	✗
BDS minima, presets	(setup ...)	✓	✗	✗	✗
Strictest effective constraints	— (max over BDS + classes)	✓	✓	✗	✓ ^h
<i>Router state</i>					
Head xy / layer / net / phase / mode	in-memory	✓	✓	✓	✓
is_dragging, state_code, routing_target	in-memory	✓	✓	✗	✗
step, step_ratio, prev-action	env-synthesized	—	✓	✓	○ ⁱ
<i>DRC</i>					
Top- <i>k</i> violation list	— (run_drc)	✓	✓ (top-32)	✓	✗ ^j
Per-net counts, error/warning split	—	✓	info only	✗	✗
<i>Action interface</i>					
6-way action mask	env-synthesized	—	info	logit mask	valid-action listing + grammar
Coordinate emission	—	—	—	discrete pointer pool	continuous text + regex grammar
Self-feedback (no-effect, streak)	—	—	✗	✗	✓ ^k

^a curves linearized once at the C++ binding; ^b segment / edge widths dropped from the textual prompt; ^c sexpr keeps xy+layer only, xml keeps full geometry; ^d shape only; ^e diameter dropped from the textual prompt; ^f via slot id, name dropped; ^g NPTH only as obstacles, unconnected pads omitted; ^h emitted as board_constraints, omitting unset fields; ⁱ history rendered as multi-turn text; ^j reward signal only, not echoed in the prompt; ^k LLM-specific (no parameter-side memory).

F Design Rule Check Catalog

Source of truth. KiCAD enumerates every Design Rule Check it can emit in `drc_item.h`: 62 distinct DRCE_* codes spanning copper-clearance, geometric (track width, annular ring, hole-to-hole), connectivity (unconnected, shorts, dangling), schematic-parity, courtyard, silkscreen, footprint-library and tuning (length / skew / diff-pair) checks. The default severity for each code is set in `board_design_settings.cpp`: every code starts at ERROR and a small set of overrides demotes 22 codes to WARNING (mostly silkscreen, schematic and library checks plus the dangling/colocated-hole pair) and 5 codes to IGNORE (courtyard membership and footprint-filter/type checks), leaving 35 stock error-level checks.

Severity surface seen by our environment. Our pipeline never edits the per-code severity table and collapses the resulting violation list into a two-way counting surface:

- **Error.** Any violation whose KiCAD severity is ERROR, together with three stock-warning codes whose presence invalidates a routed board and which we therefore count at error level: DANGLING_VIA, DANGLING_TRACK, and NET_CONFLICT.
- **Other.** Every remaining warning or ignored code: silkscreen and text overlaps, courtyard membership, library/schematic parity, colocated holes, copper slivers, isolated copper, padstack questions, and so on. These never enter the penalty or the headline DRV count.

Both the reward penalty and the evaluation DRV therefore count the same 38 checks, the 35 stock error-level checks plus the three warning-level codes above. The full enumeration with our classification is in Tables 8–9; the count-to-penalty mapping is in Table 10.

The catalog splits naturally into routing-relevant codes (the agent’s actions can produce or fix them) and non-routing codes (footprint / silk / text / library / schematic / courtyard / zone-fill issues that the agent has no actuators for). Table 8 lists the routing-relevant subset,

and Table 9 the rest. In both tables *Default* is the KiCAD stock severity and *Env* is the bucket that drives the reward penalty and the DRV count (E = Error; W = Warning; I = Ignore); Env coincides with Default for every code except DANGLING_VIA, DANGLING_TRACK, and NET_CONFLICT, which carry Default W but are counted at Env E.

Table 8: Routing-relevant DRC codes. The agent’s actions can directly produce or clean these.

ID	DRCE_* name	Settings key	Message	Default	Env
1	UNCONNECTED_ITEMS	unconnected_items	Missing connection between items	E	E
2	SHORTING_ITEMS	shorting_items	Items shorting two nets	E	E
3	ALLOWED_ITEMS	items_not_allowed	Items not allowed (custom rule)	E	E
5	CLEARANCE	clearance	Clearance violation	E	E
6	CREEPAGE	creepage	Creepage violation	E	E
7	TRACKS_CROSSING	tracks_crossing	Tracks crossing	E	E
8	EDGE_CLEARANCE	copper_edge_clearance	Board edge clearance	E	E
12	DANGLING_VIA	via_dangling	Via not / partly connected	W	E
13	DANGLING_TRACK	track_dangling	Track has unconnected end	W	E
16	HOLE_CLEARANCE	hole_clearance	Hole clearance violation	E	E
17	TRACK_WIDTH	track_width	Track width out of range	E	E
18	TRACK_ANGLE	track_angle	Track angle out of range	E	E
19	TRACK_SEGMENT_LENGTH	track_segment_length	Track segment length out of range	E	E
20	ANNULAR_WIDTH	annular_width	Annular ring too small	E	E
22	DRILL_OUT_OF_RANGE	drill_out_of_range	Hole size out of range	E	E
23	VIA_DIAMETER	via_diameter	Via diameter out of range	E	E
26	MICROVIA_DRILL_OUT_OF_RANGE	microvia_drill_out_of_range	Microvia hole out of range	E	E
32	DISABLED_LAYER_ITEM	item_on_disabled_layer	Item on disabled copper layer	E	E
37	NET_CONFLICT	net_conflict	Pad net mismatches schematic	W	E
46	ASSERTION_FAILURE	assertion_failure	Custom-rule assertion	E	E
47	GENERIC_WARNING	generic_warning	Custom-rule warning	E	E
48	GENERIC_ERROR	generic_error	Custom-rule error	E	E
50	SOLDERMASK_BRIDGE	solder_mask_bridge	Solder-mask bridges different nets	E	E
56	LENGTH_OUT_OF_RANGE	length_out_of_range	Track length out of range	E	E
57	SKEW_OUT_OF_RANGE	skew_out_of_range	Skew between tracks out of range	E	E
58	VIA_COUNT_OUT_OF_RANGE	too_many_vias	Too many / few vias on connection	E	E
59	DIFF_PAIR_GAP_OUT_OF_RANGE	diff_pair_gap_out_of_range	Diff-pair gap out of range	E	E
60	DIFF_PAIR_UNCOUPLED_LENGTH_TOO_LONG	diff_pair_uncoupled_length_too_long	Diff-pair uncoupled too long	E	E

Count-to-penalty mapping. The reward potential and the evaluation DRV count both count error-level violations, as follows.

Why this subset. Of the 62 cataloged codes, the 28 in Table 8 are routing-relevant in the sense that the agent’s actions can directly produce or fix them; the rest (Table 9) concern schematic parity, library state, courtyards, silkscreen and text, which the agent has no actuators for. Of the 28 routing-relevant codes, all but the dangling/net-conflict trio carry stock ERROR severity. The trio are stock warnings, but a dangling track or via and a net conflict each invalidate a routed board, so we count them at error level alongside the stock errors; all 28 routing-relevant codes therefore enter the penalty and the DRV count. The non-routing warnings are deliberately left out of the penalty so that training signal is concentrated on outcomes the policy can affect, and left out of the headline DRV count so that comparable boards routed by different agents are not penalized for upstream artifacts they share.

Mapping LLM failure modes to DRC buckets. Table 11 gives representative examples of how LLM routing failures surface in the DRC reports, or as non-DRC outcomes such as mask reject, parse fail, or no-effect steps. Failures that never reach the engine carry no DRC code and are bucketed as *none*.

G Reward potential and DRC penalty

This section instantiates the potential Φ defined in §3.2, which we restate here:

$$\Phi(s) = -(f_{\text{drv}}(s) + \lambda_w \ell(s) + \lambda_v n_{\text{via}}(s)), \quad (2)$$

and specifies the concrete choices used by our learned baselines: the DRC counting convention n_{drv} , the concave shape of f_{drv} , and the quality-term weights (λ_w, λ_v) .

Table 9: Non-routing DRC codes: footprint, silkscreen, text, library, schematic-parity, courtyard and zone-fill checks. Listed for completeness; none of these enter our reward penalty or DRV count.

ID	DRCE_* name	Settings key	Message	Default	Env
4	TEXT_ON_EDGE CUTS	text_on_edge_cuts	Text on Edge.Cuts layer	E	E
9	ZONES_INTERSECT	zones_intersect	Copper zones intersect	E	E
10	ISOLATED_COPPER	isolated_copper	Isolated copper fill	W	W
11	STARVED_THERMAL	starved_thermal	Thermal relief incomplete	E	E
14	DRILLED_HOLES_TOO_CLOSE	hole_to_hole	Drilled hole too close to other	W	W
15	DRILLED_HOLES_COLOCATED	holes_co_located	Drilled holes co-located	W	W
21	CONNECTION_WIDTH	connection_width	Copper connection too narrow	W	W
24	PADSTACK	padstack	Padstack questionable	W	W
25	PADSTACK_INVALID	padstack_invalid	Padstack not valid	E	E
27	OVERLAPPING_FOOTPRINTS	courtyards_overlap	Courtyards overlap	E	E
28	MISSING_COURTYARD	missing_courtyard	Footprint has no courtyard	I	I
29	MALFORMED_COURTYARD	malformed_courtyard	Malformed courtyard	E	E
30	PTH_IN_COURTYARD	pth_inside_courtyard	PTH inside courtyard	I	I
31	NPTH_IN_COURTYARD	npth_inside_courtyard	NPTH inside courtyard	I	I
33	INVALID_OUTLINE	invalid_outline	Malformed board outline	E	E
34	MISSING_FOOTPRINT	missing_footprint	Missing footprint	W	W
35	DUPLICATE_FOOTPRINT	duplicate_footprints	Duplicate footprints	W	W
36	EXTRA_FOOTPRINT	extra_footprint	Extra footprint	W	W
38	SCHEMATIC_PARITY	footprint_symbol_mismatch	Footprint attrs don't match symbol	W	W
39	FOOTPRINT_FILTERS	footprint_filters_mismatch	Footprint outside symbol filters	I	I
40	FOOTPRINT_TYPE_MISMATCH	footprint_type_mismatch	Footprint type vs. pads mismatch	I	I
41	LIB_FOOTPRINT_ISSUES	lib_footprint_issues	Footprint not in libraries	W	W
42	LIB_FOOTPRINT_MISMATCH	lib_footprint_mismatch	Doesn't match library copy	W	W
43	PAD_TH_WITH_NO_HOLE	through_hole_pad_without_hole	Through-hole pad without hole	E	E
44	FOOTPRINT	footprint	Footprint not valid	E	E
45	UNRESOLVED_VARIABLE	unresolved_variable	Unresolved text variable	E	E
49	COPPER_SLIVER	copper_sliver	Copper sliver	W	W
51	SILK_CLEARANCE	silk_over_copper	Silkscreen clipped by mask	W	W
52	SILK_EDGE_CLEARANCE	silk_edge_clearance	Silkscreen clipped by board edge	W	W
53	TEXT_HEIGHT	text_height	Text height out of range	W	W
54	TEXT_THICKNESS	text_thickness	Text thickness out of range	W	W
55	OVERLAPPING_SILK	silk_overlap	Silkscreen overlap	W	W
61	MIRRORED_TEXT_ON_FRONT_LAYER	mirrored_text_on_front_layer	Mirrored text on front layer	W	W
62	NONMIRRORED_TEXT_ON_BACK_LAYER	nonmirrored_text_on_back_layer	Non-mirrored text on back layer	W	W

Table 10: How each violation bucket enters the DRC penalty f_{drv} used in the potential Φ (Equation (2)) and the evaluation count. The training penalty f_{drv} takes the log-per-net shape of Equation (3); here x is the number of distinct nets carrying at least one in-bucket violation (breadth) and x_i is the number of in-bucket violations on net i (depth). The 3:1 ratio is the breadth-vs-depth weighting, not error-vs-warning.

Bucket	Source	Reward penalty f_{drv}	Eval DRV
Error	KiCAD ERROR severity + dangling/net-conflict trio	counted (see below)	+1
Warning	remaining KiCAD WARNING codes	0	0
Ignore	KiCAD IGNORE severity	0	0

Both sums are taken over the Error bucket (drc_errors). The CP metric reported in PCBWORLD-BENCH is the joint indicator $1\{\text{routability} = 1\} \cdot 1\{\text{drc_errors} = 0\}$. The penalty f_{drv} enters Φ with a minus sign (Equation (2)), so larger f_{drv} means smaller reward.

DRC counting convention. Let $\mathcal{V}(s)$ be the multiset of DRC violations we count at error level: the 35 stock error-level checks together with DANGLING_VIA, DANGLING_TRACK, and NET_CONFLICT, the 38 checks cataloged in Appendix F. We take $n_{\text{drv}}(s) = |\mathcal{V}(s)|$, and additionally decompose it per offending net i :

$$\begin{aligned}
 x_i(s) &= \left| \{v \in \mathcal{V}(s) : \text{net}(v) = i\} \right| & (\text{depth: per-net violation count}), \\
 x(s) &= \left| \{i : x_i(s) > 0\} \right| & (\text{breadth: number of dirtied nets}),
 \end{aligned}$$

Table 11: Representative mapping from LLM failure modes to DRC buckets. Rows whose failures never reach the engine carry no DRC code; their evidence appears in the Secondary / non-DRC column and their Bucket is *none*.

Failure mode	Primary DRC code(s)	Secondary / non-DRC	Bucket
<i>via on pad</i>	ANNULAR_WIDTH, HOLE_CLEARANCE	—	E
<i>wrong-layer endpoint</i>	DANGLING_TRACK	no-effect step	E
<i>cross-net contact</i>	SHORTING_ITEMS, CLEARANCE	—	E
<i>stranded segment</i>	DANGLING_TRACK, DANGLING_VIA	—	E
<i>detour over board edge</i>	EDGE_CLEARANCE	—	E
<i>invalid coordinate / phantom waypoint</i>	—	parse fail / mask reject	none
<i>repeated identical attempt</i>	—	rejection streak	none

so that $n_{\text{drv}}(s) = \sum_i x_i(s)$.

Shape of the DRC penalty f_{drv} . Real industrial quality requires producing boards with $\text{DRV} = 0$. A board with one residual violation is no more shippable than a board with ten, so reward improvements only matter insofar as they push the policy toward the clean-board boundary. We therefore adopt a concave, logarithmic shape for f_{drv} , which assigns a large marginal penalty to the *first* violation on a clean net and rapidly diminishing marginal penalty to each additional violation. This concentrates the learning signal on closing out the last violations, where industrial quality is decided, rather than on incremental reductions on already dirty boards. Concretely, we instantiate the concave penalty in Equation (2) as a log shape that is separately concave in breadth and depth and unbounded above:

$$f_{\text{drv}}(s) = s_{\text{agg}} \ln\left(1 + \frac{x(s)}{o}\right) + s_{\text{pn}} \sum_{i: x_i(s) > 0} \ln\left(1 + \frac{x_i(s)}{o}\right), \quad (3)$$

with hyperparameters $(s_{\text{agg}}, s_{\text{pn}}, o) = (3, 1, 2)$. The first (aggregate) term penalizes *breadth* (how many distinct nets the policy has dirtied), while the per-net sum penalizes *depth* (how many violations pile up on each individual net). The shared offset o acts as the log-curve knee, so a single new violation on a clean net contributes a bounded per-net term of $\ln(3/2)$ even when many other nets are already dirty, preventing the gradient from collapsing on heavily violated boards. The 3:1 ratio biases the policy toward routing fewer dirty nets rather than minimizing violations on a single dirty net.

Quality-term weights during training. Unless otherwise noted, the reported policies are trained with the default quality weights $(\lambda_w, \lambda_v) = (0.002, 0.1)$, so the full potential of Equation (2), including the wirelength and via terms, is optimized during training. The controllability study (§5.5) sweeps these weights over a 3×3 grid, $\lambda_w \in \{0, 0.001, 0.002\}$ and $\lambda_v \in \{0, 0.05, 0.1\}$. Its feasibility-only corner $(\lambda_w, \lambda_v) = (0, 0)$ trains against $\Phi(s) = -f_{\text{drv}}(s)$ alone.

Severity-mode coupling. The reward penalty f_{drv} , the DRC tokens emitted into the agent’s state, and the headline evaluation DRV all count the same error-level violations, so the agent is never trained against violations it cannot observe, nor evaluated on a different set than it was rewarded for. The state tokens are specified in Appendix E and the evaluation count in Appendix I.

H Wrapper Details

H.1 LLM Wrapper

The wrapper builds each prompt by instantiating a fixed template with runtime values. We partition the prompt into a static *system* message and a dynamic per-turn *user* message. The full prompt templates are reproduced in Appendix M.1. The system message contains episode-invariant content: the agent role and priority, routing guidelines, the response-format contract, the board-state schema description, and the board_static block (footprints, pads, nets, and design rules). By placing these invariant fields in the system message, the wrapper keeps the prompt prefix stable across steps, making the prompt layout compatible with prefix caching.

The user message is regenerated at every environment step from the current routing state and bookkeeping variables. It contains the 1-indexed step counter, the freshly serialized observation obtained by concatenating the routing_geometry and router_head blocks, the number of accepted actions so far, a rolling window of the recent action history, an optional rejection-streak line, and the list of action verbs allowed by the current router-phase mask. Coordinates in the dynamic observation are rounded to three decimal places in millimeters. All numerical values are normalized to a uniform decimal precision, with special sentinels (e.g., the through-hole layer) rendered as distinct tokens to prevent confusion with integers. The prompt’s guidelines cover target identification, coordinate usage, obstacle avoidance, layer-change detours, and explicit net release. The rejection slot is empty in the nominal case; after a parse failure or mask veto, consecutive identical rejections are collapsed into a single “(rejected $\times N$)” line and cleared once the next valid action is accepted.

Every model response is funneled through a strict parser before it can touch the PNS router. The parser requires a single `<think></think><action></action>` pair, a known verb, the right arity, and ASCII-only content; failures fall back to an explicit `idle` fallback at index 6. `idle` is an internal no-op index, not one of the six action types of Table 1, and leaves the board unchanged. After parsing, the action

index is intersected with the current phase’s mask (`net_select` $\rightarrow$ `start_route` $\rightarrow$ `routing`); a mask miss is also rerouted to `idle` and the offending body is pushed onto the rejection streak so the next turn’s user prompt can surface it. Successfully dispatched actions that nonetheless leave the unrouted-pin count unchanged are kept but tagged [`no effect`] in history, and any preceding `start_route` that they exposed as wasted is retroactively re-tagged. The reward consequences of each class are summarized in Table 12.

Table 12: Parser and action-mask fallback policy. The *Action taken* column is what the underlying KiCad PNS router actually executes.

Failure mode	Trigger	Action taken
Missing <action> tag	no/empty action block	idle (idx 6)
Malformed body	unknown verb, wrong arity, conv. error	idle (idx 6)
Bad <think> block	missing/duplicated reasoning channel	idle (idx 6)
Non-ASCII content	e.g. CJK characters in output	idle (idx 6)
Mask reject	valid parse, disallowed in current phase	idle (idx 6)
No-effect step	dispatched but unrouted unchanged	as emitted, tagged

H.2 RL Wrapper

The RL wrapper turns the state dictionary of §3.2 into a batched token sequence and exposes the action space of §3.2 as a typed autoregressive head over the engine-provided candidate set.

Tokenization. Each geometric or structural object becomes a single token. We support thirteen entity types: BOARD, EDGE, NET, PAD, TRACK, VIA, RAT, HEAD, four CAND_* variants for the action-time candidate pool, and DRC_VIOLATION. For a token i of type $\tau(i)$ with raw feature vector $\mathbf{f}_i$, the token embedding is the sum of an entity-type embedding and a per-type linear projection of its features:

$$\mathbf{x}_i = \mathbf{e}_{\tau(i)} + \mathbf{W}_{\tau(i)} \mathbf{f}_i.$$

Coordinates and dimensions are first normalized by the board center and a reference scale quantized onto a predefined discrete set, so identical physical quantities yield identical tokens regardless of board size. The normalized continuous fields (point coordinates, segment widths, via diameters) then pass through a sin/cos Fourier feature map with $n_{\text{freq}} = 32$ and geometric base 1.20; copper layers are encoded as the (dist_top, dist_bot) pair. TRACK tokens use symmetric endpoint pooling so that swapping the two endpoints yields the same embedding. Every geometric token also receives the head-relative distance to the current routing head as an inductive feature. A learned slot embedding distinguishes net-internal sub-objects, and the full sequence is finally normalized with LayerNorm. With $d_{\text{model}} = 128$ and $n_{\text{layers}} = 4$, the resulting flat sequence preserves the native object hierarchy of the state dictionary while remaining permutation-equivariant within each entity group.

Autoregressive action head. At step t , the action a_t of §3.2 is decoded as a triple (α, k, m) comprising an action type α , a pointer index k into the candidate set, and a routing mode m . We follow the slot table in Table 13: `net_select` and `start_route` use only a pointer; `make_line` and `make_via` use both pointer and mode; `finish` uses only mode; `net_end` uses neither. Decoding proceeds in two Transformer passes. In pass one the state sequence is extended with a single SOD token; the action-type logits are produced by a tied `action_type_head` embedding, sampled, and re-injected as a typed token to obtain a hidden state $\mathbf{h}_\alpha$. The pointer distribution scores $\mathbf{h}_\alpha$ against the per-token states of the eligible candidate slice (net tokens for `net_select`, candidate tokens otherwise) by scaled dot-product. In pass two the chosen candidate’s state token is re-emitted as `point_tok` to obtain $\mathbf{h}_k$; the routing-mode distribution then dot-products $\mathbf{h}_k$ against the routing-mode embedding table reused from the tokenizer vocabulary.

Candidate pool. The candidate set is built per-step from the active net. It contains the net’s pads (thru-hole pads expanded to one entry per copper layer), the endpoints of its already-drawn tracks, its via centers, and an eight-way directional grid of 0.5 mm offsets around the current head (a four-way grid snapped to the underlying lattice when running a D1 grid instance). Candidates are deduplicated by (x, y, layer) and truncated to 64 slots. Pointer selection therefore operates on a finite pool emitted by the engine; out-of-pool indices are masked to $-\infty$ before sampling.

Masking and stability. The same FSM-based action mask of §3.2 (`net_select` $\rightarrow$ `start_route` $\rightarrow$ `routing`) is applied to action-type logits; per-net pointer masks block out-of-scope candidates, and an additional pointer mask excludes the freshly started route’s own coordinate across all layers, preventing self-loops and the resulting performance collapse; routing-mode masks honor the configured rule set. Action-type and pointer logits are passed through $10 \cdot \tanh(\cdot)$ to prevent early-training logit blow-up. Unused slots in the emitted action triple are set to $-\text{inf}$, so every emitted action is syntactically complete and lies inside the engine’s valid-action set by construction.

Action type	Needs pointer	Needs mode
net_select	✓	
start_route	✓	
net_end		
make_line	✓	✓
make_via	✓	✓
finish		✓
idle		

Table 13: Slot usage per action type. Unused slots are emitted as -1 in the action triple and contribute no log-prob term.

I Evaluation Metrics

We evaluate PCB routing quality from multiple complementary angles, combining board-level aggregates with rollout-level statistics. For each board $b \in \mathcal{B}$, we generate k independent rollouts under the same budget; we use $k = 5$ throughout. We denote the i -th rollout for board b as $s_{b,i}$ for $i \in \{1, \dots, k\}$, and reserve $s_{b,0}$ for the bare-board state before any routing. We use $1[\cdot]$ for the indicator function.

Potential Gain (Pot.↑). Our central routing-quality measure is the *potential gain* from the bare-board state. Let $\Phi(s)$ be the board-level potential function introduced in §3.2, which jointly penalizes design rule violations, wirelength, and via count (and thereby captures net-level routing quality; see Appendix G). For a rollout $s_{b,i}$, the potential gain is

$$\Delta\Phi(s_{b,i}) = \Phi(s_{b,i}) - \Phi(s_{b,0}),$$

which measures how much routing progress has been made relative to the bare-board state $s_{b,0}$: a larger $\Delta\Phi$ reflects a cleaner, shorter, and more complete routing solution. As a reported metric, Pot. is the potential gain of the per-board selected rollout (defined next), averaged over boards:

$$\text{Pot.} = \frac{1}{|\mathcal{B}^*|} \sum_{(b, s_b^*) \in \mathcal{B}^*} \Delta\Phi(s_b^*).$$

Rollout Selection. Potential gain also defines how a single representative rollout is chosen per board, which all subsequent rollout-level metrics are computed on. For each board b we select the rollout with the largest potential gain,

$$I_b = \arg \max_{i \in \{1, \dots, k\}} \Delta\Phi(s_{b,i}), \quad s_b^* = s_{b, I_b},$$

and collect the selected pairs into

$$\mathcal{B}^* = \{ (b, s_b^*) : b \in \mathcal{B} \}.$$

Every per-board diagnostic metric defined below is reported on this potential-selected rollout s_b^* , so that all metrics describe the same representative rollout. The exceptions are Time, which is averaged over all k rollouts, and Parse-fail, which is computed from all k (see their paragraphs below).

Routability (Rout.↑). We measure connectivity using KiCAD’s *ratsnest* connections: the set of pad-to-pad connections that the netlist mandates but that have not yet been physically routed, rendered in KiCAD as the residual “rats” lines. For a rollout $s_{b,i}$, let $R(s_{b,i})$ denote the number of remaining ratsnest edges. Since $s_{b,0}$ denotes the bare-board state before any routing, the ratsnest-based routability of $s_{b,i}$ is

$$\text{rout}(s_{b,i}) = \frac{R(s_{b,0}) - R(s_{b,i})}{R(s_{b,0})} \in [0, 1],$$

i.e., the fraction of required ratsnest connections resolved relative to the bare-board state. A rollout is *fully routed* iff $R(s_{b,i}) = 0$, equivalently $\text{rout}(s_{b,i}) = 1$. We report Rout. on the per-board rollouts selected by potential gain:

$$\text{Rout.} = \frac{1}{|\mathcal{B}^*|} \sum_{(b, s_b^*) \in \mathcal{B}^*} \text{rout}(s_b^*) = \frac{1}{|\mathcal{B}^*|} \sum_{(b, s_b^*) \in \mathcal{B}^*} \frac{R(s_{b,0}) - R(s_b^*)}{R(s_{b,0})}.$$

Clean Pass (CP↑). Connectivity alone is insufficient as a measure of manufacturability. We therefore define a stricter criterion: a rollout $s_{b,i}$ is *clean* iff (i) the residual ratsnest is empty, $R(s_{b,i}) = 0$, and (ii) the number of KiCAD error-level design rule violations is zero, $\text{drv}(s_{b,i}) = 0$. Consistent with the other metrics, we evaluate cleanliness on the per-board selected rollout s_b^* (the rollout of largest potential gain):

$$\text{CP} = \frac{1}{|\mathcal{B}|} \sum_{b \in \mathcal{B}} 1[\text{rout}(s_b^*) = 1 \wedge \text{drv}(s_b^*) = 0].$$

This is the closest proxy in our evaluation to “ready-to-fabricate” output. We draw $k = 5$ rollouts per board and report CP on the selected rollout s_b^* .

DRV count (Design Rule Violation, DRV↓). We quantify the severity of rule violations by the KiCAD error count $\text{drv}(s_{b,i}) = \text{drc_errors}(s_{b,i})$. Following the main-text convention in §4.3, we report DRV on the per-board rollouts selected by potential gain:

$$\text{DRV} = \frac{1}{|\mathcal{B}^\star|} \sum_{(b,s_b^\star) \in \mathcal{B}^\star} \text{drv}(s_b^\star).$$

Physical Cost Metrics (WL↓, Via↓). We report two physical cost metrics on the per-board rollouts selected by potential gain. WL is the total routed wirelength in millimeters, summed over all nets, and Via is the number of vias in the routed board. Both are common secondary objectives in PCB routing, as shorter traces and fewer vias generally reduce routing cost and manufacturing complexity. They are aggregated over $\mathcal{B}^\star$ as

$$\text{WL} = \frac{1}{|\mathcal{B}^\star|} \sum_{(b,s_b^\star) \in \mathcal{B}^\star} \text{wl}(s_b^\star), \quad \text{Via} = \frac{1}{|\mathcal{B}^\star|} \sum_{(b,s_b^\star) \in \mathcal{B}^\star} \text{via}(s_b^\star).$$

Wallclock time (Time↓). We report wallclock routing time per rollout (in seconds), measured from the first API call of a rollout until the rollout completes or exhausts its step budget. Unlike the other metrics, Time is computed over *all* k rollouts on *every* board, regardless of routability: we take the rollout-mean per board and then the mean across boards,

$$\text{Time} = \frac{1}{|\mathcal{B}|} \sum_{b \in \mathcal{B}} \frac{1}{k} \sum_{i=1}^k \text{time}(s_{b,i}).$$

Restricting to fully routed rollouts would bias the cost in favor of methods that abort early on hard boards; the formulation above charges every rollout for the time it actually consumed. For the deterministic baselines (OrthoRoute and KRT), every rollout is identical, so @5 and @1 coincide and one routing pass yields all reported metrics. Freerouting and the RL agents report each metric as the mean over 4 seeds. The LLM agents run the five-rollout @5 protocol without seed repetition.

Parse failure rate (Parse-fail↓). Parse-fail measures whether an LLM agent’s output respects the basic KiCAD board-file syntax. Let $\text{fail}(s_{b,i}) \in \{0, 1\}$ indicate that the board produced by rollout $s_{b,i}$ fails to load as a valid .kicad_pcb and therefore cannot be evaluated. Under the default @5 protocol a board counts as a parse failure only when none of its k rollouts loads, leaving selection no candidate, whereas Parse-fail@1 is the fraction of individual rollouts that fail,

$$\text{Parse-fail} = \frac{1}{|\mathcal{B}|} \sum_{b \in \mathcal{B}} \prod_{i=1}^k \text{fail}(s_{b,i}), \quad \text{Parse-fail@1} = \frac{1}{|\mathcal{B}|k} \sum_{b \in \mathcal{B}} \sum_{i=1}^k \text{fail}(s_{b,i}).$$

Table 4 reports Parse-fail under the default @5 protocol. The panels of Figure 8 plot Parse-fail@1 in percent. The metric is most informative for the engine-free mode. It emits the routing directly in .kicad_pcb syntax, so any violation of the file grammar makes the assembled board unloadable. In the interactive and plan-and-execute modes the accepted actions execute inside the engine, which serializes the board, so the produced file is well formed by construction. We still report the metric for all three modes for comparability. Model responses that fail the per-step action parser during a rollout fall back to `idle` and leave the board unchanged (Table 12). These per-step rejections degrade CP and Rout. but are not counted by Parse-fail. Parse-fail is a diagnostic of model-generated text, so we report it only for the LLM agents.

J Experimental Setup and Model Serving

J.1 Proprietary LLM and Open-Source LLM

We benchmark the LLM agent on three OpenAI cloud-API models and one open-weights Qwen model served through Together AI. Table 14 summarizes the deployment configuration for each: serving infrastructure, hardware when exposed by the provider, weight quantization as served, reasoning mode (“hidden” for closed models with internal chain-of-thought, think-off for the Qwen variant where the explicit reasoning channel is disabled), the per-turn token budget for generation, and the datasets each model is evaluated on (D2, D3-A, and D3-B).

Table 14: Deployment configuration of every LLM in PCBWORLD-BENCH. “–” = not applicable / not exposed by the provider.

Model	Infrastructure	Hardware	Quant.	Thinking	Max tok.	Datasets
gpt-5.4	API (OpenAI)	cloud	–	hidden	256	D2, D3-A, D3-B
gpt-5.4-mini	API (OpenAI)	cloud	–	hidden	256	D2, D3-A, D3-B
gpt-5.4-nano	API (OpenAI)	cloud	–	hidden	256	D2, D3-A, D3-B
Qwen3.5-397B-A17B	API (Together)	–	FP4	think-off	512	D2, D3-A, D3-B

Table 15: Configuration of the main-result RL policy (trained on D2-train; evaluated on D2-test and zero-shot D3 test boards), shared by the PPO / PPO (terminal) / GRPO / PPO (w/o finish) entries of Table 3.

<i>Policy network (shared)</i>	
Architecture	decoder-only Transformer
d_{model} / layers / heads / FFN	128 / 4 / 8 / 512
Coordinate encoding	Fourier ($n_{\text{freq}}=32$), MLP width 128
Activation / residual	GELU / ReZero
Action / value head	pointer network (logit clip 10) / MLP critic (PPO only)
<i>Optimization (shared)</i>	
Optimizer	AdamW ($\epsilon=10^{-5}$, weight decay 10^{-4})
Learning rate	10^{-4} (20-iteration warmup)
Discount γ / GAE λ	0.995 / 0.95
PPO clip ϵ	0.2
Entropy / value coef.	0.01 / 0.5
Max gradient norm	0.5
Batch size / update epochs	256 / 4
Parallel envs / rollout horizon	32 / 512
Episode step limit	256
<i>Reward (shared)</i>	
Wirelength weight λ_w / via weight λ_v	0.002 / 0.1
DRC penalty f_{drv}	log-per-net, error-level violations
<i>Per-variant</i>	
PPO	dense per-step reward, 300 iterations
PPO (terminal)	sparse terminal reward, 300 iterations
GRPO	sparse reward, group size 16, 1800 iterations
PPO (w/o finish)	as PPO, with <code>finish</code> removed from the agent’s action space
Seeds / hardware	42–45 / 1× NVIDIA L40 48 GB

K Hyperparameters and compute

This appendix records the compute infrastructure, the configuration of the main-result RL policies (Table 15), and the optimization and architecture settings of the D1 grid-size/action-abstraction scalability experiment (Table 16).

K.1 Compute infrastructure

All RL agents were trained on a single NVIDIA L40 48 GB GPU per run. Each training node carries two AMD EPYC 9354 32-core CPUs and 1.5 TiB of RAM and runs Ubuntu 20.04 or 22.04. Evaluation rollouts execute headless KiCAD engine calls and are CPU-bound. The environment and the PPO and GRPO agents run in Python 3.12.13 with PyTorch 2.8.0 (CUDA 12.8, cuDNN 9.10.2), NumPy 2.2.6, and Gymnasium 1.2.3. The PPO and GRPO training loops are our own implementation. The engine is KiCAD 9.0.8 with the `kicad-python` 0.6.0 Python $\leftrightarrow$ C++ binding. Freerouting 2.1.0 runs under OpenJDK 21.0.6, and OrthoRoute is pinned to commit `f45dc68`. The A2C (Jumanji) and Sable baselines run in a separate JAX 0.5.3 environment with Jumanji 1.1.1, Mava 0.2.0, Flax 0.10.3, and Optax 0.2.8.

K.2 Main-result RL policy hyperparameters

The PPO, PPO (terminal), GRPO, and PPO (w/o finish) entries of Table 3 all train the same decoder-only Transformer policy from scratch on D2-train, selecting checkpoints on D2-valid; the policy is then evaluated by rollout on D2-test and the zero-shot D3 open-source set. The variants share the architecture, optimizer, and reward weights (Table 15), and differ only in the reward form (per-step dense vs. terminal sparse), the number of training iterations, and, for PPO (w/o finish), the removal of `finish` from the agent’s action space. In practice we train with $\gamma = 0.995$ (Table 15), for which the telescoping equivalence between the per-step and terminal rewards (§3.2) holds approximately.

K.3 D1 grid-size scalability learned-policy hyperparameters

The D1 scalability experiment compares PPO against grid-action A2C (Jumanji) and Sable on matched Jumanji-Connector v2 (Connector-v2) tasks: PPO consumes KiCAD board files, while the grid-action baselines consume fixed Connector-v2 NPZ instances exported from the same splits. Table 16 lists the training settings that materially affect the RL results, with one hyperparameter family per row. Final reporting evaluates each selected checkpoint on the exact 128 test boards per grid and seed. The main-text figure (Figure 6) reports the single-rollout

Rout.@1; the @5 diagnostics of Table 23 draw five rollouts per board and retain the selected rollout before aggregation. For the grid-action baselines, training uses `train.npz`, training-time validation uses `val.npz`, and the final reported metric is recomputed on `test.npz`. This split policy is important because the Connector-v2 NPZ files are exported from the same generated KiCAD board splits used by the PPO agent. The main figure reports the routability trend; Table 23 retains the per-seed routability breakdown. For those diagnostics, A2C (Jumanji) and Sable wirelength is converted from grid steps to millimeters using the grid pitch, 100/G mm per cell.

Table 16: D1 learned-policy hyperparameters. RL settings for the learned methods. Each row isolates one training, architecture, or reward setting; “–” denotes a setting that is not used by that method.

Setting	PPO	A2C (Jumanji)	Sable
<i>Data and budget</i>			
Grids	10, 50, 100, 200, 500	10, 50, 100, 200; 500 OOM	10, 50, 100, 200, 500
Seeds	42–45	42–45	42–45
Train split	10K KiCAD boards/grid	10K Connector-v2 NPZ instances/grid	10K Connector-v2 NPZ instances/grid
Eval protocol	128 exact test boards/grid $\times$ 5 rollouts; selected rollout/board	128 exact test NPZ instances/grid $\times$ 5 rollouts; selected rollout/board	128 exact test NPZ instances/grid $\times$ 5 rollouts; selected rollout/board
Episode horizon	256	256	256
Native endpoint	300 PPO iterations	D1-10: 8300 epochs; D1-50: 1600 epochs; D1-100: 5000 epochs; D1-200: 2200 epochs	D1-10: 1.58M updates; D1-50: 1.76M; D1-100: 1.46M; D1-200: 0.8125M; D1-500: 0.18M
<i>Optimization</i>			
Parallelism	32 envs	total batch 256	16 envs
Rollout length	512	10	128
Update epochs	4 PPO epochs	100 learner steps/epoch	4 PPO epochs
Minibatch/update batch	minibatch 256	total batch 256	update batch 2; 2 minibatches
Learning rate	10^{-4} with 20-iter warmup	2×10^{-4} for D1-10/D1-50; 1.25×10^{-5} for D1-100; 6.25×10^{-6} for D1-200	actor lr 2.5×10^{-4}
Discount γ	0.995	1.0	0.99
GAE / trace	GAE $\lambda = 0.95$	bootstrap factor 0.95	GAE $\lambda = 0.95$
Policy clip	0.2	–	0.2
Entropy coefficient	0.01	0.01	0.01
Value coefficient	0.5	TD loss weight 1.0	0.5
Gradient clip	0.5	–	0.5
<i>Architecture</i>			
Observation	KiCAD token stream	Connector grid tensor	vector Connector observation
Backbone	token Transformer	Jumanji Connector A2C	feed-forward Sable with retention memory
Width	$d_{\text{model}} = 128$, FFN 512	conv channels 32, MLP 512	embedding 64
Depth / heads	4 layers, 8 heads	4 encoder blocks, 8 heads, key size 16	1 block, 1 head
Action constraint	action masking with same-net bias	grid-action mask	grid-action policy
<i>Reward and accounting</i>			
Reward form	newly connected nets – movement cost	newly connected nets – movement cost	newly connected nets – movement cost
Movement unit	wirelength increment in mm	grid-action step	grid-action step
Movement cost	$0.003 \times \Delta \text{WL}(\text{mm})$	D1-10: 0.03; D1-50: 0.006; D1-100: 0.003; D1-200: 0.0015	D1-10: 0.03; D1-50: 0.006; D1-100: 0.003; D1-200: 0.0015; D1-500: 0.0006
Extra step cost	0	–	–
WL accounting unit	native mm	grid steps $\times$ 100/G mm	grid steps $\times$ 100/G mm

K.4 D1 training-time validation budgets

The D1 learning curves are diagnostic W&B validation logs, not the test-board rollout protocol used for the main results. The PPO runs use the 300-iteration checkpoint batch and complete in 11.6–22.0 hours per seed/grid in W&B runtime (Table 17). The A2C (Jumanji) and Sable sweeps use fixed native endpoints chosen to match roughly one day of active training for the corresponding grid scale. Table 17 records these endpoints so that the cropped axes in Figure 22 are auditable.

Table 17: D1 W&B validation-log endpoints. Runtime is W&B _runtime, summarized as mean [min,max] hours across the listed seeds. The native endpoint is the largest logged training coordinate used by the validation curves. The displayed endpoint is the plotted coordinate after the figure’s display-only clipping/scaling; for Sable this is the logged update counter divided by 100.

Method	Subset	Seeds	Runtime (h)	Native endpoint	Displayed endpoint	State / note
PPO	D1-10	42–45	19.5 [17.7, 22.0]	300 iter	300	finished
PPO	D1-50	42–45	14.0 [13.7, 14.3]	300 iter	300	finished
PPO	D1-100	42–45	13.3 [12.3, 15.9]	300 iter	300	finished
PPO	D1-200	42–45	13.0 [11.6, 15.7]	300 iter	300	finished
PPO	D1-500	42–45	13.7 [12.0, 16.7]	300 iter	300	finished
A2C (Jumanji)	D1-10	42–45	approx. 24	8300 epochs	5000 (clipped)	finished
A2C (Jumanji)	D1-50	42–45	approx. 24	1600 epochs	1600	finished
A2C (Jumanji)	D1-100	42–45	approx. 24	5000 epochs	5000	finished
A2C (Jumanji)	D1-200	42–45	approx. 24	2200 epochs	2200	finished
A2C (Jumanji)	D1-500	–	–	–	–	OOM
Sable	D1-10	42–45	approx. 24	1.58M updates	15.8k	finished
Sable	D1-50	42–45	approx. 24	1.76M updates	17.6k	finished
Sable	D1-100	42–45	approx. 24	1.46M updates	14.6k	finished
Sable	D1-200	42–45	approx. 24	0.8125M updates	8.125k	finished
Sable	D1-500	42–45	approx. 24	0.18M updates	1.8k	finished

K.5 Training-time validation curves for D1 grid-size scalability

Figure 22 visualizes the validation diagnostics logged during training for the learned policies in Figure 6. These curves are not the final reporting protocol: the main figure evaluates the selected checkpoints on the exact 128 test boards per grid and seed. The plots instead show optimization dynamics under each method’s native training axis, with seed mean and standard-deviation bands.

K.6 Baseline Router Hyperparameters

We compare our learned policy against three rule-based routers: KiCadRoutingTools (KRT), OrthoRoute (a PathFinder-style negotiation router), and Freerouting 2.1.0. Each router consumes its own preferred input format, so every input board is converted accordingly (Freerouting: DSN, OrthoRoute: ORP, KRT: native KiCAD PCB) before the router is invoked, and the hyperparameters used for each board are recorded alongside the routing result. All settings are taken from each router’s *default* configuration; no tuning is performed. The *Tunable* column indicates whether a parameter is freely user-configurable (✓) or hard-coded inside the router (✗).

Table 18: KiCadRoutingTools (KRT) hyperparameters. Search-algorithm parameters, cost-function weights, and geometry parameters are all exposed as CLI arguments of `route.py` and are therefore user-configurable. Geometry parameters are set to match the design rules of each input board.

Group	Parameter	Value	Tunable
Search	grid_step (mm)	0.1	✓
	max_iterations	200,000	✓
	max_probe_iterations	5,000	✓
	max_rip_up_count	3	✓
	heuristic_weight	1.9	✓
	ordering_strategy	mps	✓
Cost	via_cost	50	✓
	via_proximity_cost	10	✓
	turn_cost	1,000	✓
	direction_preference_cost	50	✓
Geometry	layers	{F.Cu, B.Cu}	✓
	track_width (mm)	matched to input board’s design rules	✓
	clearance (mm)	matched to input board’s design rules	✓
	via_size (mm)	matched to input board’s design rules	✓
	via_drill (mm)	matched to input board’s design rules	✓

Grid resolution and design rules in OrthoRoute. PathFinder does not explicitly evaluate trace width or clearance; the grid spacing acts as an implicit minimum trace-to-trace distance. We use the upstream default `grid_pitch` = 0.4 mm, and we do *not* enforce the board’s design rule

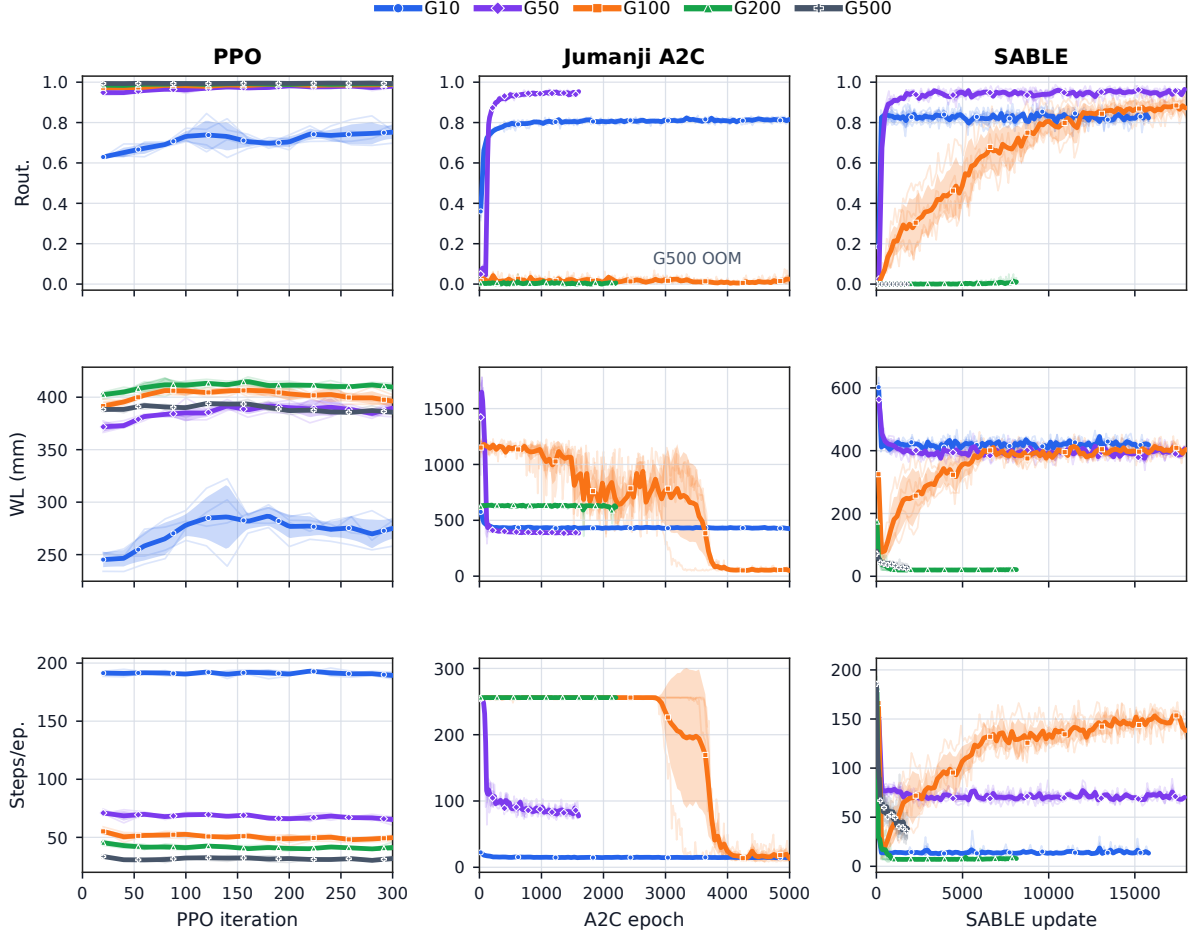


Figure 22: D1 grid-size scalability training-time validation diagnostics. Columns separate PPO, A2C (Jumanji), and Sable runs; rows show routability, wirelength, and episode length. This four-seed plot uses the Connector-v2 NPZ split protocol and native endpoints; the A2C (Jumanji) axis is displayed up to 5000 epochs and the Sable update counter is divided by 100 for readability. Thick lines are seed means, translucent bands show one standard deviation, and faint traces show individual seeds. These curves are diagnostic W&B validation logs; the reported metrics are recomputed separately on the test-board rollouts.

(track_width + clearance) on grid_pitch. grid_pitch is not exposed through the tool’s interface, so changing it requires modifying the source code. A single global pitch also cannot express boards whose netclass settings assign different clearances to different nets, as in the curated D3 boards. Honoring per-net clearances in a uniform-grid router would require changing the routing algorithm itself rather than adjusting a parameter. PathFinder also internally tunes a subset of its parameters (pres_fac_*, hist_*, max_iterations) on the fly based on per-board flexibility.

A note on iteration semantics. The three routers use mutually incomparable termination conditions: KRT’s max_iterations counts A* expansions across all rip-up attempts, OrthoRoute’s max_iterations counts PathFinder negotiation rounds, and Freerouting’s -mp sets the maximum number of full ripup-and-reroute passes. We therefore do not normalize these budgets and instead use each router’s default termination condition.

L Additional results

Per-seed std for D2/D3-A/D3-B routing quality. Table 21 reports mean±std for the evaluation that the main Table 3 summarizes with mean only, extended to the per-metric DRV/WL/Via breakdown.

Table 19: OrthoRoute hyperparameters. PathFinder-style negotiation router [39]. The first block (*Exposed*) lists parameters that the user can change directly; the second block lists internal parameters that are fixed at the PathFinderConfig defaults. We use the upstream defaults for all values.

Group	Parameter	Value	Tunable
Exposed	max_iterations	250	✓
	use_gpu	True	✓
PathFinder defaults	grid_pitch (mm)	0.4	✗
	pres_fac_init	1.0	✗
	pres_fac_mult	1.1	✗
	pres_fac_max	64.0	✗
	hist_gain	0.2	✗
	hist_cost_weight	10.0	✗
	base_cost_weight	0.3	✗
	portal_discount	0.4	✗
	span_alpha	0.15	✗

Table 20: Freerouting 2.1.0 hyperparameters. We list only the CLI options that we explicitly set. Other options exposed by the JAR (e.g. `-mt` for the multi-threading level, `-pp` for the number of postroute passes) are left at their JAR defaults.

Group	Parameter	Value	Tunable
CLI	jar version	freerouting-2.1.0.jar	✓
	<code>-mp</code> (max passes)	10	✓
	<code>-Xmx</code> (JVM heap)	4 GiB	✓

Single-rollout (@1) counterparts. Table 22 rescores the same evaluation under the @1 protocol. Every rollout is scored individually and each entry averages the per-rollout scores, so no best-of-five selection is applied. Time is averaged over all five rollouts under both protocols, so it is unchanged from Table 21 and is omitted. The method ordering of Table 21 is largely preserved, with only swaps among closely tied methods. The margins widen, however, because the gain from selection varies widely across methods. PPO and PPO (w/o finish) stay close to their @5 scores on D2, with CP moving from 1.00 to 0.98 and 1.00, whereas PPO (terminal) drops from 0.92 to 0.61 and GPT-5.4-mini falls from 0.58 to 0.20. The same pattern holds zero-shot, where PPO (w/o finish) keeps a D3-A CP of 0.81 against 0.94 under @5 while the smaller LLM backbones lose about half of their clean passes. Removing selection therefore separates methods that appear comparable under @5 without reordering them. The @5 gains also show that the stochastic methods convert additional rollouts into better boards through selection, whereas the deterministic routers return a single solution that repeated runs cannot improve. Since final board quality outweighs a constant factor of routing computation in EDA practice, spending a larger computation budget for a better solution is a practical advantage.

Grid-action baseline (Jumanji). Jumanji [5] instantiates the simplest form of PCB routing under the grid-action abstraction: each net is an agent stepping on a neighbor grid in four directions from a source pad to a target pad, and any visited cell becomes unusable for all agents (single-occupancy). The episode reward is the number of connected nets minus λ times the total wirelength. With every net configured as a 2-pad connection of unit-cell width, we adopt the grid-action agents trained in this environment, A2C (Jumanji) and Sable, as baselines and compare them against the PPO agent on identical instances.

KiCAD-API treatment of grid scale. Under the grid-action abstraction, enlarging the grid preserves the local four-neighbor move set but expands both the state space and the routing horizon, so the same physical instance becomes a strictly harder MDP. The KiCAD-API abstraction sidesteps this by treating track width and clearance as *environment parameters* rather than as state: the agent issues geometric net-segment commands whose action dimensionality is independent of the grid resolution, and grid scaling only changes the routing margin available to each net. The D1 grid-size scalability experiment isolates exactly this design choice on otherwise identical instances.

Quantitative breakdown and horizon argument. Our KiCAD-API agent’s CP rises from 0.63 on D1-10 to 1.00 from D1-50 to D1-500 (selected-rollout routability 0.90 to 1.00; the single-rollout Rout.@1 in Figure 6 is correspondingly lower, e.g. 0.77 on D1-10), tracking the geometric easing of the instance as each unit-cell-width net occupies a smaller fraction of the board. The grid-action baselines display the opposite profile: A2C (Jumanji) and Sable are competitive on D1-10 and D1-50, but A2C collapses by D1-100 and runs out of memory at D1-500, while Sable remains viable at D1-100 (CP 0.73) but collapses at D1-200 and D1-500. This is not a lack of wall-clock effort. The grid-action baselines consume far more environment steps through JAX/vectorized training. Rather, the experiment isolates the core abstraction gap. Finer grids make the physical board easier, but they lengthen the local-move MDP horizon that grid-action agents must explore, whereas the KiCAD-API action space keeps the decision horizon tied to routed segments rather than cells.

Table 21: Routing-quality std breakdown (D2 / D3-A / D3-B). Companion to the main Table 3, adding DRV, WL (mm), and Via alongside CP, Pot., Rout., and Time. Freerouting and the RL agents report mean±sample standard deviation ($n-1$) over 4 seeds. Deterministic methods (OrthoRoute, KiCadRoutingTools) are run once, and the LLM agents run the five-rollout @5 protocol without seed repetition, so no seed-level std applies to either. All metrics are computed on the potential-selected rollout, except Time, which is averaged over all five rollouts.

Split	Method	CP↑	Pot.↑	Rout.↑	DRV↓	WL↓	Via↓	Time↓
D2	Freerouting	1.00 ± 0.00	15.47 ± 0.07	1.00 ± 0.00	0.00 ± 0.00	393.9 ± 1.0	2.32 ± 0.03	2.69 ± 0.01
	OrthoRoute	0.01	0.38	0.34	8.54	323.4	11.30	2.54
	KiCadRoutingTools	1.00	15.52	1.00	0.00	373.9	8.09	0.82
	GPT-5.4	0.96	16.05	1.00	0.05	401.4	1.61	94.04
	GPT-5.4-mini	0.58	12.45	0.86	1.40	370.8	2.07	33.93
	GPT-5.4-nano	0.55	12.23	0.85	1.45	364.6	1.91	63.39
	Qwen3.5-397B	0.33	9.77	0.74	2.48	325.0	1.56	47.58
	PPO	1.00 ± 0.00	16.24 ± 0.01	1.00 ± 0.00	0.00 ± 0.00	423.8 ± 2.8	1.70 ± 0.08	0.43 ± 0.04
	GRPO	1.00 ± 0.00	14.03 ± 1.15	1.00 ± 0.00	0.00 ± 0.00	581.2 ± 52.4	6.31 ± 3.12	1.16 ± 0.13
	PPO (terminal)	0.92 ± 0.03	15.61 ± 0.17	0.98 ± 0.01	0.16 ± 0.05	432.9 ± 4.5	1.63 ± 0.11	1.37 ± 0.09
	PPO (w/o finish)	1.00 ± 0.00	16.21 ± 0.01	1.00 ± 0.00	0.00 ± 0.00	439.4 ± 6.1	1.48 ± 0.08	0.88 ± 0.06
D3-A	Reference	1.00	23.16	1.00	0.00	170.2	1.18	–
	Freerouting	0.80 ± 0.01	22.71 ± 0.09	0.91 ± 0.00	0.86 ± 0.05	156.2 ± 0.2	0.74 ± 0.06	7.09 ± 0.26
	OrthoRoute	0.02	-6.16	0.53	99.70	129.8	26.97	2.20
	KiCadRoutingTools	0.74	20.00	0.94	2.38	142.8	3.69	0.65
	GPT-5.4	0.65	19.42	0.91	1.31	150.8	2.09	231.15
	GPT-5.4-mini	0.28	14.22	0.72	3.88	127.0	0.39	56.85
	GPT-5.4-nano	0.30	14.16	0.73	3.84	129.1	0.45	100.76
	Qwen3.5-397B	0.34	14.88	0.75	3.63	121.0	0.47	81.82
	PPO	0.86 ± 0.03	21.46 ± 0.20	0.95 ± 0.01	0.89 ± 0.09	165.4 ± 1.3	0.92 ± 0.29	1.83 ± 0.26
	GRPO	0.85 ± 0.04	18.83 ± 1.50	0.98 ± 0.01	0.46 ± 0.15	279.0 ± 41.4	9.39 ± 4.52	3.54 ± 0.82
	PPO (terminal)	0.82 ± 0.02	21.02 ± 0.41	0.95 ± 0.01	0.67 ± 0.09	171.3 ± 5.7	2.61 ± 1.62	2.58 ± 0.22
	PPO (w/o finish)	0.94 ± 0.02	21.78 ± 0.10	0.99 ± 0.00	0.21 ± 0.07	224.1 ± 7.2	2.16 ± 0.65	3.05 ± 0.15
D3-B	Reference	1.00	63.96	1.00	0.00	570.3	6.70	–
	Freerouting	0.78 ± 0.05	61.06 ± 0.57	1.00 ± 0.00	10.60 ± 0.35	535.4 ± 3.3	3.48 ± 0.22	9.94 ± 2.17
	OrthoRoute	0.00	-10.25	0.44	188.20	370.0	39.80	9.30
	KiCadRoutingTools	0.20	40.94	0.86	44.70	472.3	11.80	3.27
	GPT-5.4	0.00	34.72	0.62	17.30	347.1	3.60	865.83
	GPT-5.4-mini	0.00	30.84	0.61	17.60	561.5	0.90	422.06
	GPT-5.4-nano	0.00	30.66	0.59	18.60	460.0	0.30	510.25
	Qwen3.5-397B	0.00	25.99	0.51	21.20	313.6	5.00	222.29
	PPO	0.45 ± 0.10	50.20 ± 4.35	0.85 ± 0.06	8.28 ± 2.95	668.6 ± 64.1	6.17 ± 1.77	10.77 ± 3.51
	GRPO	0.10 ± 0.00	30.00 ± 6.99	0.69 ± 0.09	15.82 ± 4.66	1147.6 ± 210.7	24.50 ± 9.42	11.20 ± 5.12
	PPO (terminal)	0.38 ± 0.05	44.33 ± 4.28	0.81 ± 0.06	10.18 ± 2.90	667.4 ± 20.2	11.78 ± 3.40	12.60 ± 1.72
	PPO (w/o finish)	0.42 ± 0.13	46.45 ± 1.36	0.87 ± 0.01	7.85 ± 0.81	1000.5 ± 126.4	10.20 ± 2.02	14.44 ± 1.69

Per-seed std for D1 grid-size scalability. Companion to Figure 6 in the main text: per-seed mean±std for the learned D1 methods on the exact 128 test boards per grid and seed. The main figure reports the single-rollout Rout.@1; Table 23 additionally keeps clean-pass, potential-gain, and wirelength diagnostics on the selected rollout. Each board is evaluated with five rollouts, then one selected rollout (largest potential gain) is retained before aggregation. CP requires full connectivity with zero error-level DRVs on this selected rollout; routability divides routed nets by the five two-pin nets in each Connector-v2 board. WL is absolute routed wirelength in millimeters; A2C (Jumanji) and Sable

Table 22: Single-rollout (@1) routing quality (D2 / D3-A / D3-B). Companion to Table 21 under the @1 protocol of §4.3. Every rollout is scored without selection and each entry averages the per-rollout scores. For the stochastic methods, $\pm$ is the sample standard deviation ($n-1$) across replicates, where a replicate is one rollout of one seed (20 replicates for Freerouting and the RL agents, 5 for the LLM agents, which run a single pass without seed repetition), whereas the $\pm$ of Table 21 is seed-level. Deterministic methods (OrthoRoute, KiCadRoutingTools) are run once, so @1 and @5 coincide. Time is averaged over all five rollouts under both protocols and is reported in Table 21. Best CP and Pot. per split (excluding the Reference row) are in bold.

Split	Method	CP $\uparrow$	Pot. $\uparrow$	Rout. $\uparrow$	DRV $\downarrow$	WL $\downarrow$	Via $\downarrow$
D2	Freerouting	1.00 $\pm$ 0.00	14.42 $\pm$ 0.07	1.00 $\pm$ 0.00	0.00 $\pm$ 0.01	409.8 $\pm$ 2.1	2.48 $\pm$ 0.04
	OrthoRoute	0.01	0.38	0.34	8.54	323.4	11.30
	KiCadRoutingTools	1.00	15.52	1.00	0.00	373.9	8.09
	GPT-5.4	0.88 $\pm$ 0.02	15.24 $\pm$ 0.12	0.98 $\pm$ 0.00	0.27 $\pm$ 0.04	411.8 $\pm$ 2.5	2.00 $\pm$ 0.08
	GPT-5.4-mini	0.20 $\pm$ 0.03	6.46 $\pm$ 0.45	0.49 $\pm$ 0.03	4.62 $\pm$ 0.27	208.6 $\pm$ 14.5	0.96 $\pm$ 0.10
	GPT-5.4-nano	0.24 $\pm$ 0.03	6.97 $\pm$ 0.44	0.54 $\pm$ 0.03	4.21 $\pm$ 0.27	233.3 $\pm$ 11.3	1.25 $\pm$ 0.09
	Qwen3.5-397B	0.14 $\pm$ 0.04	5.81 $\pm$ 0.40	0.48 $\pm$ 0.02	4.64 $\pm$ 0.19	229.6 $\pm$ 10.2	1.02 $\pm$ 0.06
	PPO	0.98 $\pm$ 0.02	15.84 $\pm$ 0.11	0.99 $\pm$ 0.01	0.06 $\pm$ 0.05	458.4 $\pm$ 9.0	2.22 $\pm$ 0.16
	GRPO	0.95 $\pm$ 0.02	11.43 $\pm$ 1.15	0.98 $\pm$ 0.02	0.19 $\pm$ 0.14	744.3 $\pm$ 113.9	9.84 $\pm$ 3.57
	PPO (terminal)	0.61 $\pm$ 0.07	12.06 $\pm$ 0.80	0.80 $\pm$ 0.06	1.60 $\pm$ 0.53	380.2 $\pm$ 33.8	1.62 $\pm$ 0.51
	PPO (w/o finish)	1.00 $\pm$ 0.00	15.81 $\pm$ 0.09	1.00 $\pm$ 0.00	0.01 $\pm$ 0.01	497.1 $\pm$ 10.6	1.93 $\pm$ 0.20
	Reference	1.00	23.16	1.00	0.00	170.2	1.18
	Freerouting	0.75 $\pm$ 0.01	22.04 $\pm$ 0.15	0.90 $\pm$ 0.00	1.08 $\pm$ 0.08	156.6 $\pm$ 1.0	0.86 $\pm$ 0.05
	OrthoRoute	0.02	-6.16	0.53	99.70	129.8	26.97
	KiCadRoutingTools	0.74	20.00	0.94	2.38	142.8	3.69
D3-A	GPT-5.4	0.42 $\pm$ 0.03	16.15 $\pm$ 0.39	0.81 $\pm$ 0.02	2.71 $\pm$ 0.20	148.1 $\pm$ 5.7	2.36 $\pm$ 0.17
	GPT-5.4-mini	0.13 $\pm$ 0.01	8.97 $\pm$ 0.20	0.49 $\pm$ 0.01	7.16 $\pm$ 0.18	81.9 $\pm$ 4.0	0.21 $\pm$ 0.05
	GPT-5.4-nano	0.15 $\pm$ 0.01	8.35 $\pm$ 0.42	0.47 $\pm$ 0.02	7.46 $\pm$ 0.30	83.0 $\pm$ 6.1	0.31 $\pm$ 0.07
	Qwen3.5-397B	0.19 $\pm$ 0.01	10.98 $\pm$ 0.43	0.59 $\pm$ 0.02	5.75 $\pm$ 0.31	102.7 $\pm$ 2.2	0.47 $\pm$ 0.08
	PPO	0.68 $\pm$ 0.03	19.22 $\pm$ 0.38	0.87 $\pm$ 0.02	1.91 $\pm$ 0.20	164.9 $\pm$ 3.9	1.09 $\pm$ 0.37
	GRPO	0.64 $\pm$ 0.07	15.07 $\pm$ 1.82	0.90 $\pm$ 0.03	1.67 $\pm$ 0.52	337.1 $\pm$ 53.6	14.13 $\pm$ 5.17
	PPO (terminal)	0.65 $\pm$ 0.03	18.23 $\pm$ 0.65	0.86 $\pm$ 0.02	1.95 $\pm$ 0.29	171.0 $\pm$ 7.5	3.50 $\pm$ 1.92
	PPO (w/o finish)	0.81 $\pm$ 0.03	19.55 $\pm$ 0.40	0.94 $\pm$ 0.01	0.95 $\pm$ 0.26	262.7 $\pm$ 9.9	3.87 $\pm$ 1.03
	Reference	1.00	63.96	1.00	0.00	570.3	6.70
	Freerouting	0.64 $\pm$ 0.09	58.86 $\pm$ 0.84	0.99 $\pm$ 0.01	11.94 $\pm$ 0.79	533.5 $\pm$ 7.3	3.88 $\pm$ 0.37
	OrthoRoute	0.00	-10.25	0.44	188.20	370.0	39.80
	KiCadRoutingTools	0.20	40.94	0.86	44.70	472.3	11.80
	GPT-5.4	0.00 $\pm$ 0.00	25.65 $\pm$ 1.33	0.44 $\pm$ 0.02	24.88 $\pm$ 1.10	240.3 $\pm$ 21.8	2.06 $\pm$ 0.73
	GPT-5.4-mini	0.00 $\pm$ 0.00	23.96 $\pm$ 0.85	0.49 $\pm$ 0.02	23.00 $\pm$ 1.14	408.6 $\pm$ 56.3	0.94 $\pm$ 0.27
	GPT-5.4-nano	0.00 $\pm$ 0.00	23.62 $\pm$ 2.73	0.47 $\pm$ 0.06	24.00 $\pm$ 2.39	363.9 $\pm$ 45.7	0.36 $\pm$ 0.21
	Qwen3.5-397B	0.00 $\pm$ 0.00	19.02 $\pm$ 0.93	0.41 $\pm$ 0.02	26.84 $\pm$ 0.74	249.6 $\pm$ 15.9	6.24 $\pm$ 3.77
D3-B	PPO	0.21 $\pm$ 0.14	37.60 $\pm$ 6.62	0.65 $\pm$ 0.11	17.33 $\pm$ 4.61	518.8 $\pm$ 94.9	5.06 $\pm$ 1.72
	GRPO	0.07 $\pm$ 0.04	21.76 $\pm$ 6.35	0.55 $\pm$ 0.09	22.28 $\pm$ 4.40	1054.5 $\pm$ 169.8	27.26 $\pm$ 8.97
	PPO (terminal)	0.15 $\pm$ 0.07	33.47 $\pm$ 3.82	0.64 $\pm$ 0.06	17.54 $\pm$ 2.75	550.0 $\pm$ 53.3	10.64 $\pm$ 4.01
	PPO (w/o finish)	0.22 $\pm$ 0.08	35.99 $\pm$ 4.06	0.72 $\pm$ 0.07	15.32 $\pm$ 3.17	1008.6 $\pm$ 138.4	14.57 $\pm$ 4.71
	Reference	1.00	63.96	1.00	0.00	570.3	6.70
	Freerouting	0.64 $\pm$ 0.09	58.86 $\pm$ 0.84	0.99 $\pm$ 0.01	11.94 $\pm$ 0.79	533.5 $\pm$ 7.3	3.88 $\pm$ 0.37
	OrthoRoute	0.00	-10.25	0.44	188.20	370.0	39.80
	KiCadRoutingTools	0.20	40.94	0.86	44.70	472.3	11.80

wirelength is converted from grid steps using the grid pitch, 100/G mm per cell. WL values with Rout. < 0.5 are retained for auditability but are not comparable compactness estimates because incomplete episodes measure only produced traces.

Per-seed std for PPO/GRPO reward-training analysis. Per-seed mean $\pm$ std for PPO, GRPO, and PPO (terminal) is part of Table 21. Table 24 reports the board-level timing distribution.

Checkpoint provenance for reward analysis. The reward sweep in Figure 9 uses the existing 9-cell $\times$ 4-seed checkpoint set from the PCBWORLD-BENCH experiments; it is not a new training sweep. The default PPO (per-step) cell with wire penalty 0.002, via penalty 0.1, and seed 42 uses the best checkpoint from a completed same-configuration recovery run because the original symbolic-link target for that seed

Table 23: D1 grid-size sweep across action abstractions. Mean $\pm$ std across seeds (per-board rollout selected by Pot.) on the 128 Connector-v2 test boards per grid. Metrics are clean pass (CP), potential gain (Pot.), routability (Rout.), and routed wirelength (WL, in mm; grid-action baselines converted from grid steps via the grid pitch 100/G mm/cell). A2C (Jumanji) on D1-500 exceeded memory (OOM). WL at Rout.< 0.5 reflects only partial traces and is not a comparable compactness estimate.

Split	Method	CP $\uparrow$	Pot. $\uparrow$	Rout. $\uparrow$	WL $\downarrow$
D1-10	PPO	0.63 $\pm$ 0.01	4.75 $\pm$ 0.02	0.90 $\pm$ 0.00	344.2 $\pm$ 5.2
	A2C (Jumanji)	0.51 $\pm$ 0.02	4.22 $\pm$ 0.12	0.87 $\pm$ 0.01	407.1 $\pm$ 6.5
	Sable	0.46 $\pm$ 0.02	4.04 $\pm$ 0.11	0.86 $\pm$ 0.01	382.3 $\pm$ 2.7
D1-50	PPO	1.00 $\pm$ 0.00	6.92 $\pm$ 0.03	1.00 $\pm$ 0.00	370.1 $\pm$ 2.6
	A2C (Jumanji)	0.91 $\pm$ 0.00	6.51 $\pm$ 0.03	0.98 $\pm$ 0.00	462.2 $\pm$ 6.3
	Sable	0.88 $\pm$ 0.02	6.38 $\pm$ 0.11	0.97 $\pm$ 0.01	370.9 $\pm$ 3.0
D1-100	PPO	1.00 $\pm$ 0.00	6.95 $\pm$ 0.00	1.00 $\pm$ 0.00	363.0 $\pm$ 1.2
	A2C (Jumanji)	0.00 $\pm$ 0.00	-5.47 $\pm$ 0.15	0.03 $\pm$ 0.02	76.7 $\pm$ 6.6
	Sable	0.73 $\pm$ 0.13	5.59 $\pm$ 0.71	0.94 $\pm$ 0.04	379.1 $\pm$ 7.2
D1-200	PPO	1.00 $\pm$ 0.00	6.92 $\pm$ 0.02	1.00 $\pm$ 0.00	368.7 $\pm$ 8.5
	A2C (Jumanji)	0.00 $\pm$ 0.00	-5.72 $\pm$ 0.02	0.01 $\pm$ 0.00	156.7 $\pm$ 1.8
	Sable	0.00 $\pm$ 0.00	-5.69 $\pm$ 0.09	0.01 $\pm$ 0.01	25.7 $\pm$ 1.6
D1-500	PPO	1.00 $\pm$ 0.00	6.95 $\pm$ 0.00	1.00 $\pm$ 0.00	358.1 $\pm$ 1.5
	A2C (Jumanji)	OOM			
	Sable	0.00 $\pm$ 0.00	-5.79 $\pm$ 0.00	0.00 $\pm$ 0.00	28.4 $\pm$ 6.9

was unavailable. All reward-analysis metrics are still recomputed with the same selected-rollout and KiCAD evaluator protocol used for the other seeds.

Training-time validation curves for PPO/GRPO analysis. Figure 23 reports the W&B diagnostics logged while training the three PPO/GRPO analysis policies. As with the D1 scalability curves, these curves are included to show optimization behavior, not to define the reported metrics. Main Table 3 and Table 21 use the selected @5 rollout artifacts, with quality metrics recomputed from saved PCBs using the KiCAD evaluator. We do not plot a training-time CP curve here because the logged validation scalars are not consistently the CP quantity used in the tables. CP requires the joint condition of full connectivity and zero KiCAD error-level DRVs on the same selected route.

Table 24: Episode timing distribution for the RL agents. Board-level latency distribution for the Time entries in main Table 3. Timing uses the recorded per-board rollout time and includes board reload/reset, policy rollout, and routed-PCB serialization. D3-A excludes 0096_karabas-nano_wifi_revA, the one board that cannot be loaded through the engine interface (Appendix O.5).

Dataset	Configuration	Boards	Mean	Median	P95	Max
D2	PPO	128	0.43	0.37	0.92	1.42
	GRPO	128	1.16	0.94	2.94	3.67
	PPO (terminal)	128	1.37	1.20	2.96	3.85
	PPO (w/o finish)	128	0.88	0.74	1.82	3.16
D3-A	PPO	99	1.83	1.04	5.39	15.00
	GRPO	99	3.54	3.09	8.55	12.98
	PPO (terminal)	99	2.58	1.88	5.91	8.78
	PPO (w/o finish)	99	3.05	1.90	8.44	24.62
D3-B	PPO	10	10.77	9.31	18.23	19.98
	GRPO	10	11.20	10.44	14.76	15.15
	PPO (terminal)	10	12.60	10.96	22.49	26.08
	PPO (w/o finish)	10	14.44	15.23	19.48	20.82

PPO is usually fast on D3-A (median 1.04 s, mean 1.83 s), with the tail driven by a few hard real boards that run to the 256-step horizon and trigger expensive KiCAD geometry updates during rollout and serialization. On the larger D3-B boards all four configurations run several times slower (means 10.8–14.4 s, three to six times their D3-A means).

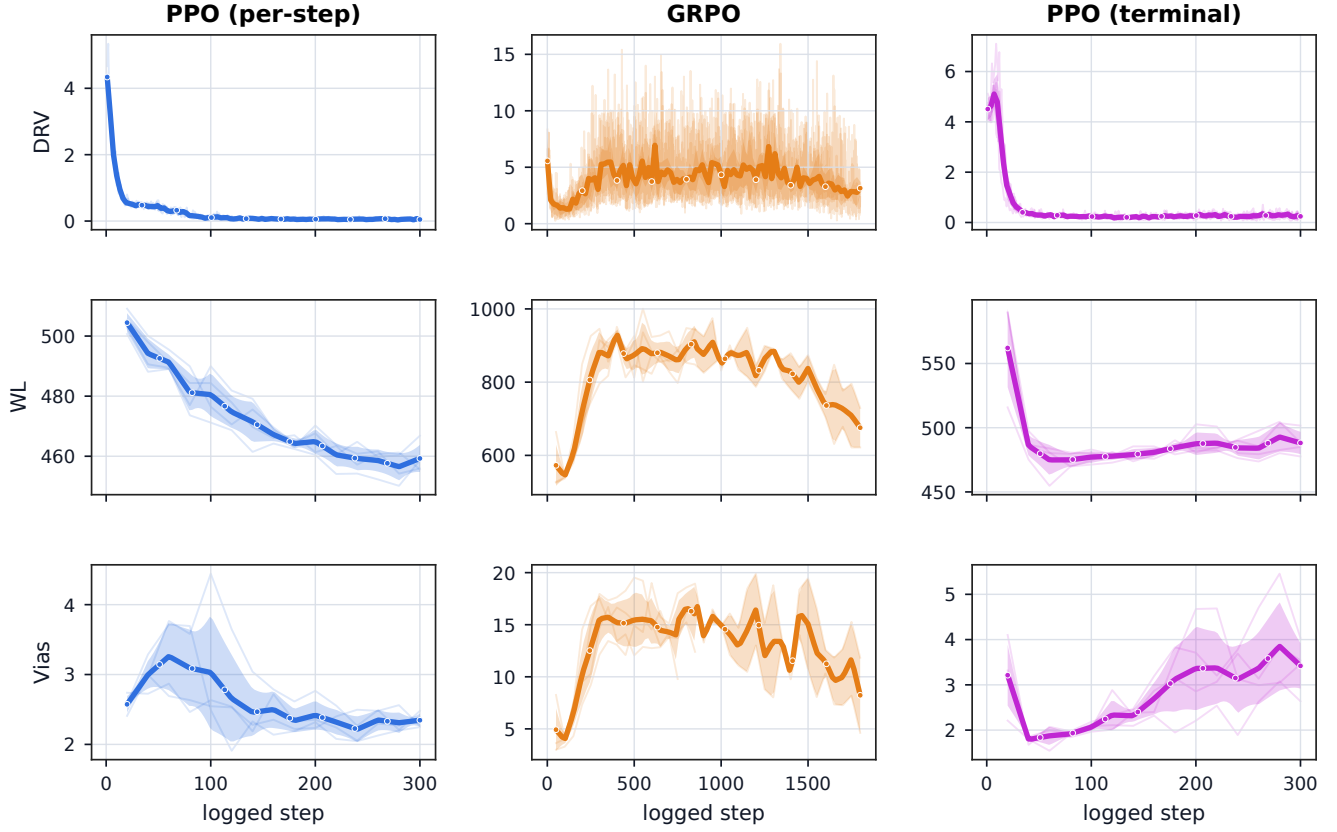


Figure 23: PPO/GRPO training-time validation diagnostics. Columns correspond to PPO (per-step), GRPO, and PPO (terminal). Rows show DRV, wirelength, and via diagnostics. Curves are W&B logged values over the logged training step or evaluation iteration; thick lines are seed means, translucent bands show one standard deviation, and faint traces show individual seeds. Final Table 3 CP and quality values are computed separately from the selected routed PCBs.

L.1 Throughput and Token Usage in LLM Experiments

For the OpenAI LLM runs, we record wall-clock time and token consumption per episode. Tables 25–27 report the per-episode means: latency in seconds, and three token streams, *System* (system-prompt tokens accumulated across all turns), *User* (user-message tokens, dominated by the per-step state-of-board observation), and *Response* (model-generated tokens, including any hidden reasoning the API counts on the input/output side). *Total* is the sum of the three.

Table 25: Per-episode latency and token usage on D2 (128 boards $\times$ 5 rollouts = 640 episodes). Tokens are means over completed episodes.

Model	Sec/ep	System	User	Response	Total
GPT-5.4	94.0	62,899	39,909	3,823	106,631
GPT-5.4-mini	33.9	51,538	26,438	2,382	80,358
GPT-5.4-nano	63.4	66,140	38,124	4,260	108,524

M LLM prompts

M.1 PCBWORLD agent

Each step’s prompt is assembled from the template in Figure 24, partitioned into the static system message and the per-step user message as described in Appendix H.1. Placeholders with curly-braces are populated at runtime from the environment state.

Table 26: Per-episode latency and token usage on D3-A (99 boards $\times$ 5 rollouts = 495 episodes). Tokens are means over completed episodes.

Model	Sec/ep	System	User	Response	Total
GPT-5.4	231.1	263,612	152,583	8,683	424,879
GPT-5.4-mini	56.9	156,258	57,796	3,525	217,578
GPT-5.4-nano	100.8	199,424	92,200	6,667	298,291

Table 27: Per-episode latency and token usage on D3-B (10 boards $\times$ 5 rollouts = 50 episodes). Tokens are means over completed episodes.

Model	Sec/ep	System	User	Response	Total
GPT-5.4	865.8	1,119,161	775,289	26,717	1,921,167
GPT-5.4-mini	422.1	1,147,573	918,588	20,134	2,086,296
GPT-5.4-nano	510.3	1,117,479	861,073	29,945	2,008,496

Placeholders. The runtime values used throughout the experiments in this paper are:

- `{StateFormatDesc}` – schema description of the board-state encoding. We use the S-expression encoding (`state_format=sexpr`); the alternative XML encoding is supported but not used in the reported runs.
- `{CurrentStep}` – 1-indexed step counter, ranging from 1 to the per-episode budget T . We set $T = 200$ for D2, D3-A, and D3-B.
- `{CurrentObservation}` – concatenation of the `routing_geometry` and `router_head` blocks, regenerated every step; all floating-point coordinates are rounded to 3 decimal places (mm).
- `{ValidStepCount}` – number of accepted (parsed *and* mask-valid) actions issued so far in the episode; bounded above by `{CurrentStep}`.
- `{ActionHistory}` – rolling window of the most recent `history_length = 2` turns, each rendered as a `<think>/<action>` pair; entries that left the unrouted-pin count unchanged are prefixed with `[no effect]`.
- `{RejectedAttempts}` – conditional slot, empty string in the nominal case. When the previous action was rejected (parse failure or mask veto) it expands to a single line of the form `Step F-L: <body> (rejected  $\times N$ )`; consecutive identical bodies are collapsed and the streak is cleared by the next valid action.
- `{ValidActions}` – the action verbs allowed by the current router phase mask (`net_select  $\rightarrow$  start_route  $\rightarrow$  routing`); the parser’s index set has seven entries, the six action types plus the internal `idle` fallback at index 6 (Appendix H.1).

You are a PCB routing agent for KiCad. Connect unconnected pins (points in `routing_geometry`) within each net. Never connect pins from different nets.

Priority

1. Complete all connections (reduce unconnected points to zero).
2. Avoid DRC violations (no crossing other nets, respect clearance).
3. Minimize total wirelength.

Routing Guidelines

- Targets are the `(point ...)` entries in `routing_geometry`; the final segment must land on the target pad’s `(x, y)` while the cursor is on that pad’s layer (points carry no layer -- look up the pad in `board_static` or the nearby tracks).
- Use exact coordinates from the observation (pad positions, point targets, track endpoints). Only invent coordinates for detour waypoints.
- Avoid static obstacles, board edges, and existing tracks on the same layer (any net, including the current one).
- When a path is blocked, detour: switch layers with ``make_via`` (offset ≥ 1 mm from any pad -- never place a via on a pad), route around on the opposite layer, then switch back.
- When the cursor is already near the remaining target with a clear path, prefer ``finish <mode>`` over a manual ``make_line``.

Layer-change detour pattern

Route from `(0, 0)` on layer 1 to `(10, 10)` on layer 1, bypassing an obstacle via layer 2:

1. `start_route 0.0 0.0 1`
2. `make_via 0.0 0.5 w` # offset off the source pad, then cross to layer 2
3. `start_route 0.0 0.5 2` # re-enter routing on the new layer
4. `make_via 10.0 9.5 w` # STOP before the target’s `(x, y)` and cross back
5. `start_route 10.0 9.5 1` # re-enter routing on the target pad’s layer
6. `make_line 10.0 10.0 w` # final segment lands on the pad, on its own layer

Do NOT draw or end on a pad from the wrong layer

```

Never let `make_line` or `make_via` reach the target pad's exact (x, y) while the cursor is on a different layer than that pad. If you do,
you are trapped:
- placing a via at the pad violates the "no via on pad" rule, and
- the pad is unreachable from the wrong layer, so `finish` will fail repeatedly.
Always make the layer-return `make_via` at an offset waypoint (>=1mm away from the target pad) *before* the track arrives at the target's (x
, y).

# Board State
{StateFormatDesc}

## Step {CurrentStep}
{CurrentObservation}

# History (step {CurrentStep}, {ValidStepCount} valid actions taken)
{ActionHistory}{RejectedAttempts}

# Valid Actions
Choose exactly one from:
{ValidActions}

# Response Format
Output exactly one <think>...</think> followed by one <action>...</action> with a valid action. Never repeat think-action pairs.
"[no effect]" in history = action made no progress; "Last rejected attempt" below = action was refused. Do not repeat either.

```

Figure 24: Full prompt template for the PCBWORLD agent.

M.2 Baselines

Engine-free (open-loop) generation. An LLM directly generates the routed board in KICAD board syntax: given the initial board state, it emits the wire segments and vias that are inserted into the .kicad_pcb file.

```

You are an expert PCB routing engineer using KiCad PCB format.

Your task is to generate valid PCB routing (tracks and vias) for a given KiCad PCB board.

## Instructions
- Analyze the given PCB layout, including components, pads, and nets.
- Generate routing (segments and vias) that correctly connects pads belonging to the same net.
- Ensure:
  - No short circuits between different nets
  - Minimal via usage unless necessary
  - Reasonable routing paths (avoid unnecessary detours)
  - Respect layer usage (F.Cu, B.Cu)

## Routing geometry: OCTILINEAR ROUTING with the 45-degree-ONLY constraint (mandatory)
Use octilinear routing for every `(segment ...)` . Octilinear routing is a wire/edge routing style where connections are restricted to eight directions: the four cardinal (horizontal, vertical) plus the four diagonals at 45 degrees. It sits between rectilinear routing (4 directions, Manhattan-style) and fully arbitrary Euclidean routing.

### The 45-degree-only constraint (sub-rule of octilinear)
Whenever a segment is not horizontal or vertical, it MUST be a strict 45 (or 135) degree diagonal -- i.e. exactly  $|dx| == |dy|$ . No other diagonal angle is permitted. The following are all VIOLATIONS:
30, 60      (e.g. (0,0) -> (10, 5.77) or (0,0) -> (5, 8.66))
22.5, 67.5   ("half-octilinear" angles)
13.16, 26.6  (or any other arbitrary slope)
Even tiny rounding deviations break the rule: `(start 0 0) (end 10 9.9)` is NOT octilinear. If you intend a 45-degree diagonal, the rise must equal the run exactly.

### Allowed segment shapes (and only these)
Concretely, a segment from `(start sx sy)` to `(end ex ey)` is octilinear iff one of the following holds, with `dx = ex - sx` and `dy = ey - sy`:
1. dy == 0      (East / West -- 0, horizontal)
2. dx == 0      (North / South -- 90, vertical)
3. dx == dy (and both nonzero) (NE / SW -- 45, diagonal)
4. dx == -dy (and both nonzero) (NW / SE -- 135, diagonal)
Equivalently, every segment is horizontal, vertical, or a 45-degree diagonal where  $|dx| == |dy|$ . Anything else is forbidden.

```

```

### How to handle non-octilinear paths
If the natural route between two pads is at an angle that is not in {0, 45, 90, 135} degrees, decompose it into multiple octilinear segments
joined at corners. For example, to go from (0,0) to (10,3) you might use:
(segment ... (start 0 0) (end 3 3) ...) ; 45-degree diagonal
(segment ... (start 3 3) (end 10 3) ...) ; horizontal
Use as many segments as needed; each one must individually satisfy the rule above.

This is exactly the constraint KiCad's interactive PNS router enforces with `corner_mode = MITERED_45`.

## Output Format
- Return ONLY the routing additions in valid KiCad PCB format.
- Do NOT repeat the full board.
- Only include `(segment ...)` and `(via ...)` entries.

```

Figure 25: Full prompt template for the engine-free (open-loop) generation.

Plan-and-execute (open-loop) generation. An LLM emits the full PCBWorld action sequence ahead of execution, without observing any intermediate board state during generation.

```

You are an expert PCB routing engineer driving a KiCad routing API.

You will be given the *initial* state of a PCB board (footprints, pads, nets) and must output the **complete sequence of routing API calls**
that would route every net.

## API
The router exposes 6 high-level actions. Emit one per line.
net_select <net_id>           Select a net to start routing.
start_route <x_mm> <y_mm> <layer> Begin routing at a pad position.
                                layer: 1 = F.Cu (top), 2 = B.Cu (bottom).
make_line <x_mm> <y_mm> <mode>   Extend a track to (x,y) on the current layer.
make_via <x_mm> <y_mm> <mode>   Extend track to (x,y), drop a via, switch layer.
                                AFTER make_via, the current layer flips.
                                Auto-complete the active route on the *CURRENT*
                                layer to the nearest pending pad of the
                                current net. CANNOT cross layers – if the
                                remaining pad is on a different layer, finish
                                will NOT reach it.
finish <mode>
                                Mark the active net as fully routed.
net_end
mode is one of: m (MarkObstacles), p (PushAndShove), w (Walkaround). Use w by default.

## Reading <BOARD>
The input is KiCad-style sexpr. Inside `(nets ...)` each net lists its pads:
(pad <id> <x> <y> <layer_tag>)
Layer tags:
`1` -> the pad lives on layer 1 (F.Cu, top).
`2` -> the pad lives on layer 2 (B.Cu, bottom).
`th` -> through-hole, electrically present on BOTH layers; you
      may treat it as either 1 or 2 when choosing start_route's
      layer parameter or as the via-side of a make_via.

## Coordinates and layers (MUST follow exactly)
- Copy each pad's `` and `` token **verbatim** – same digits, same decimal places, no rounding. The router only accepts a pad's *exact*
  * (x_mm, y_mm). Example: if the board has
    (pad D0 67.500 44.700 th)
  then output
    start_route 67.500 44.700 1
  not `start_route 67.5 44.7 1`.
- start_route MUST be issued at a pad position of the currently selected net.

## Layer-choice rule for the FIRST pad
You pick a `` argument for `start_route`. The choice determines which layer the router head sits on, which constrains every
subsequent action.

Pick the start layer using THIS priority:
1. If the first pad has tag `1` -> use 1.

```

```
2. If the first pad has tag `2` -> use 2.
3. If the first pad has tag `th` and at least one other pad of
   this net has tag `1` or `2` -> use that other pad's layer.
   (This avoids an unnecessary via.)
4. If the first pad has tag `th` and all other pads also `th`
   -> use 1.

## Routing protocol (state machine – violations cause the entire net to fail)
After `net_select`, the router is in {has_net=True, is_routing=False}.
After a SUCCESSFUL `start_route`, it moves to {has_net=True, is_routing=True}.
`make_line`, `make_via`, `finish` are ONLY valid while is_routing=True.
`net_end` closes the net and returns to {has_net=False}.
Emitting any of make_line / make_via / finish / net_end before a
successful start_route causes the entire net to fail.

## Per-topology emission rules (CASE BY CASE – match the net's shape)
Resolve each pad's *effective* layer using the layer-choice rule (`th` resolves to either 1 or 2 by neighbour). Then dispatch:

### Case A – 0 or 1 pads : skip the net entirely.

### Case B – 2 pads on the SAME effective layer L
net_select <id>
start_route <P1.x> <P1.y> L
make_line <P2.x> <P2.y> <mode> ; recommended over finish
net_end

(alternative: `finish <mode>` also works – auto-completes head to P2 on L)

### Case C – 2 pads on DIFFERENT layers (cross-layer)
`finish` cannot cross layers. Pick an intermediate point P1' and go
P1 -> P1' -> P2, using make_via at P1' to switch layers:
net_select <id>
start_route <P1.x> <P1.y> L1 ; L1 = P1's layer
make_via <P1'.x> <P1'.y> <mode> ; switch L1 -> L2 at P1'
start_route <P1'.x> <P1'.y> L2 ; L2 = P2's layer, restart at P1'
make_line <P2.x> <P2.y> <mode> ; (or `finish <mode>`)
net_end

### Case D – k pads (k >= 3) all on the SAME effective layer L
net_select <id>
start_route <P1.x> <P1.y> L
make_line <P2.x> <P2.y> <mode>
start_route <P2.x> <P2.y> L
make_line <P3.x> <P3.y> <mode>
... up to P(k-1) ...
make_line <P(k).x> <P(k).y> <mode> ; (or `finish <mode>`)
net_end

## Anti-patterns (do NOT do these)
✗ `finish` immediately after start_route when the second pad is
  on a different layer. finish will route within the current
  layer and silently miss the cross-layer pad.
  ✓ Use make_via to land on the cross-layer pad (Case C).
✗ Picking a `` for start_route that no pad of this net
  lives on (e.g. layer 2 when both pads are on 1).
  ✓ Apply the layer-choice rule.
✗ Reformatting coordinates (5.7 instead of 5.700, dropping
  trailing zeros). The board's pad table has the exact form;
  copy it.
✗ Emitting `make_line <Pk.x> <Pk.y>` THEN `finish` for the
  final pad of a same-layer chain. `finish` already handles Pk
  – the explicit make_line creates a duplicate / overlapping
  segment and may break DRC.
✗ Forgetting `net_end` between nets. The next `net_select`
  fails if the previous net wasn't closed.
✗ Calling start_route at coordinates that are not a pad of the
  currently selected net (e.g. picking a pad of a different
  net by accident). The route fails silently.
```

```
## Output Format
Wrap the entire sequence in <actions>...</actions>. One action per line. No commentary inside the block. Do NOT repeat the board.
```

Figure 26: Full prompt template for the plan-and-execute (open-loop) generation.

N Visualization gallery and LLM failure traces

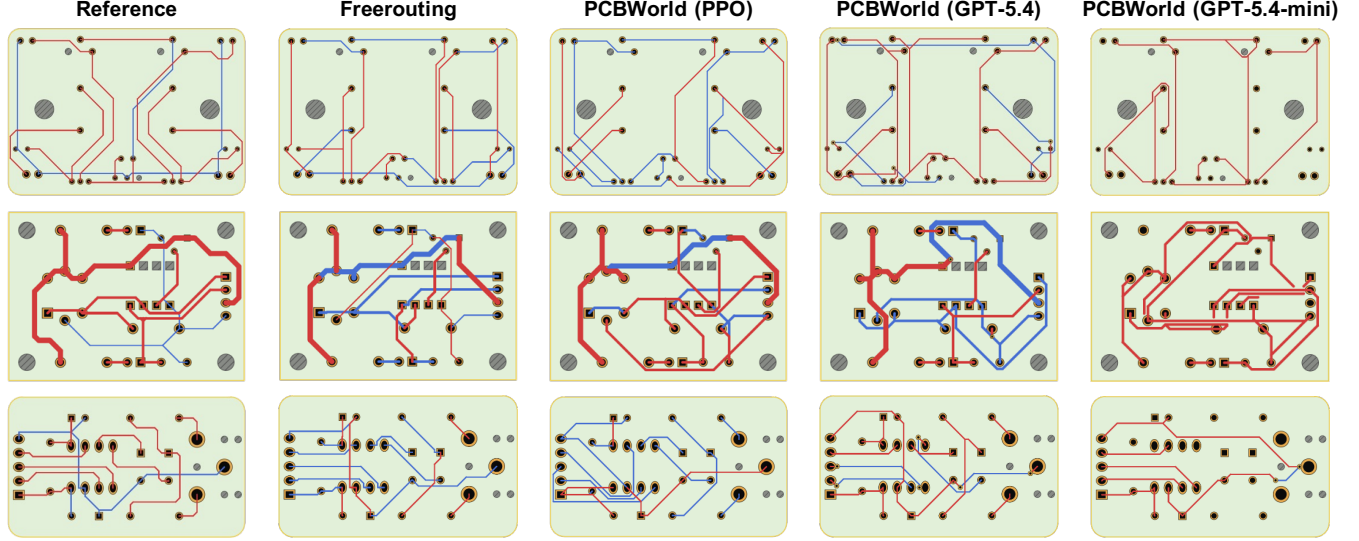


Figure 27: Examples of routed boards (D3-A). Full version of the main-text Figure 7, showing all three boards. Red and blue traces mark wire segments on the two copper layers.

Importance of net selection order. Across the studied rollouts, `net_select`, nominally a simple routing-target selection action, emerges as a critical strategic decision whose first invocation often determines whether a board can be successfully routed (see Figure 28). In the `0018_hy_adapter` case, successful episodes consistently begin with selecting a routing order that minimizes early routing difficulty, whereas unsuccessful episodes choose a more challenging net at the outset and subsequently fail to recover within the limited step budget. Since the agent rarely performs corrective recovery behaviors after an unfavorable decision, the initial `net_select` effectively constrains the future routing space and strongly influences the overall routing outcome. These observations suggest that routing-target ordering is a primary factor governing downstream routing success.

Importance of route editing. The agent’s action space (Table 1) contains no operation that edits committed copper. PCBWORLD does expose track-removal APIs (`delete_track_*`, `delete_via_*`) among the Action methods cataloged in Appendix C, but we exclude them from the agent’s action space to keep it compact and to scope the task to routing itself. The agent can push existing traces aside through the `push_n_shove` routing mode, yet it cannot delete or redraw a trace it has already committed. As a consequence, the agent tends to struggle when severe congestion occurs, since it has limited capability to resolve routing deadlocks after they emerge. Nevertheless, we observe that successful routing remains achievable when the agent proactively avoids congestion through early-stage planning and anticipatory reasoning.

Figure 29 analyzes successful and failed cases on (`0100_smt-zvs-driver_IH10-mc`). Since the action space does not include direct editing of pre-existing tracks, once congestion is detected, the agent tends to repeatedly attempt rerouting through alternative paths. In contrast, when the agent proactively anticipates potential congestion and initiates routing from a different region in advance, it can achieve more effective routing with fewer steps.

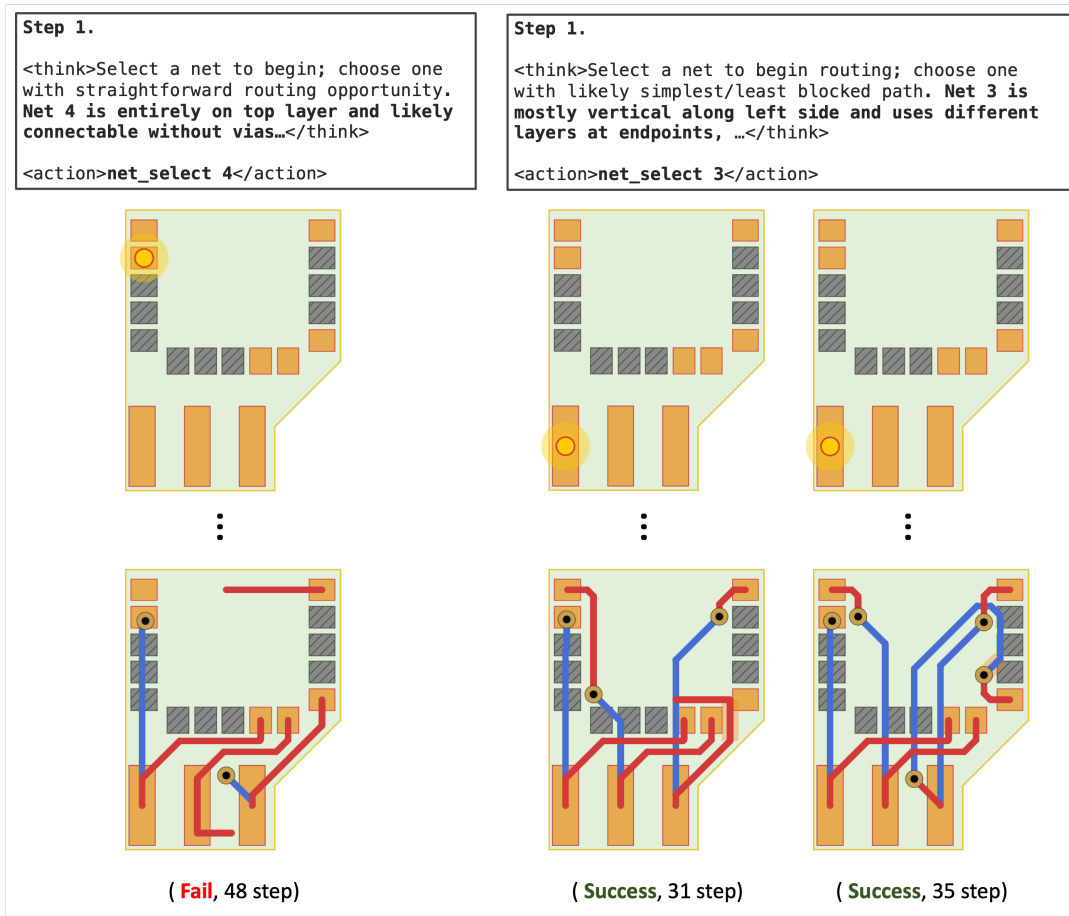


Figure 28: Case Study 1 (0018_hy_adapter). In this action design, determining the routing priority of nets is critically important. Although different nets were selected for different reasons, we observe that routing succeeds when a particular net (net 3) is selected first, whereas it fails when another net (net 4) is selected first.

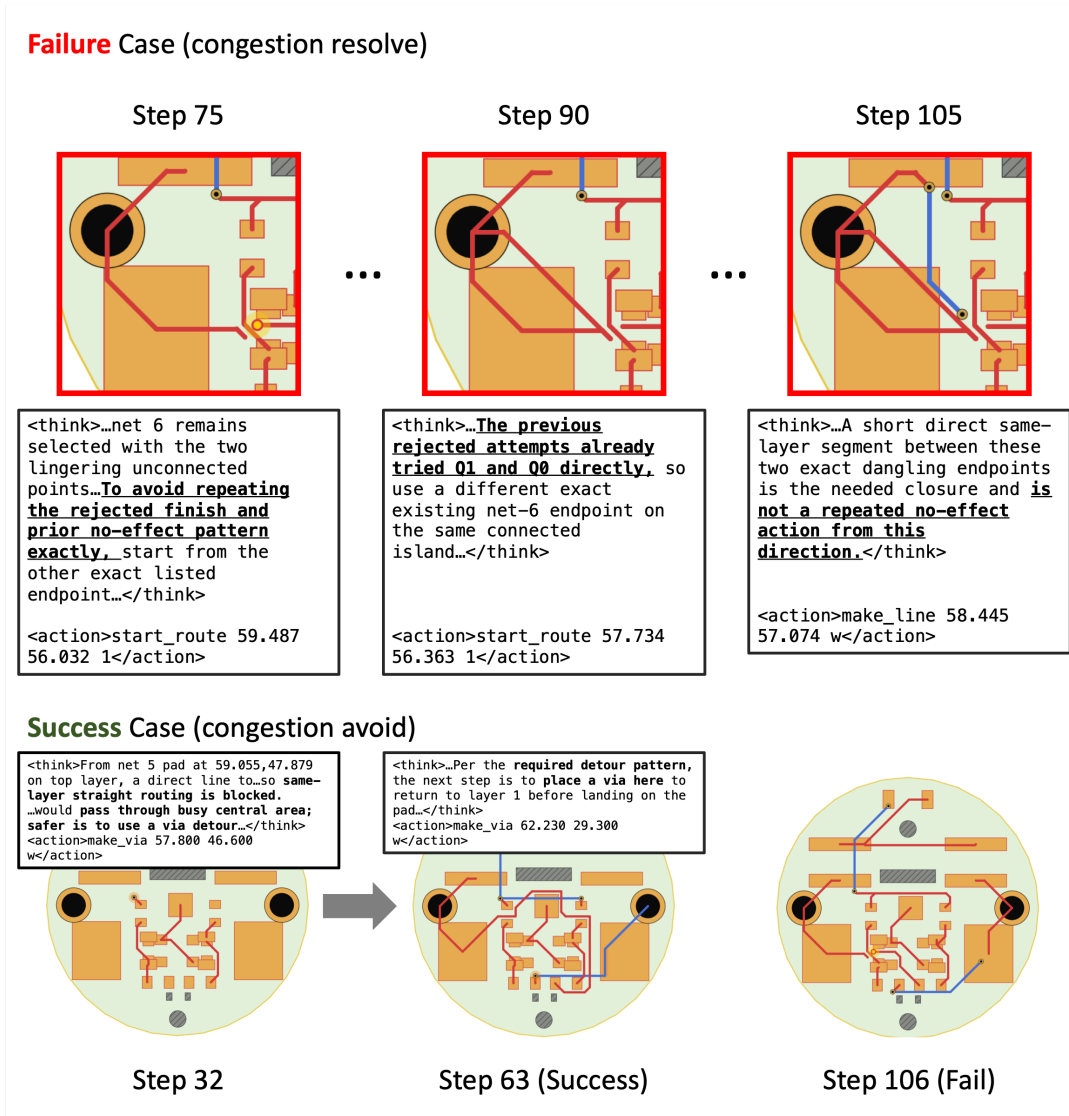


Figure 29: Case Study 2 (0100_smt-zvs-driver_IH10-mc). Failure case (top) and success case (bottom). Since the action space does not include direct editing of pre-existing tracks, once congestion is encountered, the agent repeatedly attempts rerouting through alternative paths. In contrast, proactively anticipating congestion and initiating routing from a different region enables more effective routing with fewer steps.

O Datasheet for Datasets

This appendix summarizes PCBWORLD-BENCH using the main categories of the Datasheets for Datasets template [13]. The benchmark is a collection of KiCAD-native PCB routing instances used with the engine-grounded evaluation protocol described in §4.

O.1 Motivation

PCBWORLD-BENCH was created to evaluate PCB routing methods scored by the same engine-checked evaluator. DRC status, connectivity, wirelength, and via count are all computed by KiCAD. The dataset targets comparison among tool-using LLM agents, RL agents, grid-action baselines, and rule-based routers, rather than supervised imitation of a single reference solution. It contains no human-subject records or personal data.

O.2 Composition

Each instance is stored as a `.kicad_pcb` file containing the board geometry, placed pads/components, nets, design rules, and the initial routing state. The benchmark has three task families: D1 is a 1-layer grid-routing family converted from Jumanji Connector-style instances [5]; D2 is the Synthetic Gridless Board family, a 2-layer multi-pin family with configurable net and pad counts; and D3 is a 679-board curated set derived from PCBench’s open-source PCB corpus [17], normalized to the KiCAD 9 format, populated with each board’s stored design rules, and retained only when fully connected and DRV-free. D1 and D2 each provide 10,000/128/128 train/valid/test instances (D2-train/D2-valid/D2-test for D2), while D3 is reserved for zero-shot evaluation and stratified by pad count into D3-A (Small), D3-B (Medium), and D3-C (Large). Table 2 summarizes the per-family statistics; the stratification rule, the per-split sizes, and the boards used for evaluation are given in Appendix O.5.

O.3 Collection Process

D1 is produced by converting grid instances into executable KiCAD boards. D2 is generated from a parametric synthetic board generator. D3 starts from PCBench, a public PCB routing dataset with per-board `raw.kicad_pcb` and processed `kicad_pcb` files, `PCB-RDL.final.json` routing descriptions, metadata, data augmentation scripts, and an RL environment [17]. We use the KiCAD-9-converted `PCBs_version_9` pool as the raw D3 source and apply the compatibility filtering below. No crowdworkers or paid participants were involved.

O.4 Preprocessing

The D3 preprocessing step addresses KiCAD-version DRC compatibility rather than changing PCB geometry. The `PCBs_version_9` source pool contains 1,182 boards converted to KiCAD 9; many were originally authored in KiCAD 4–7, so conversion assigns newer KiCAD 9 default DRC constraints that can flag violations even when the physical board file is unchanged. We therefore apply a project-file compatibility pass to `.kicad_pro` files only: non-routing manufacturing/courtyard checks such as `solder_mask_bridge`, `drill_out_of_range`, `malformed_courtyard`, and `courtyards_overlap` are ignored, and KiCAD-9 floor constraints for annular width, copper-edge clearance, and hole clearance are relaxed to match earlier-version board compatibility. Core routing constraints used for evaluation, including clearance, shorts/crossings, track width, via diameter, and the source-board netclass settings, are preserved. Track width is always present in the source settings; when a board stores no explicit clearance value, the clearance is instead set to the minimum spacing observed among its own routed traces.

After this compatibility pass, 679 boards remain in the curated D3 set. The pass does not edit the physical `.kicad_pcb` content: a byte-level comparison confirmed that all 679 retained board files are unchanged before and after preprocessing. The changes are limited to DRC rule interpretation in the project file, and the retained boards are revalidated through the same KiCAD engine interface used for evaluation.

O.5 Splits and Evaluation Subsets of D3 Boards

Ordering and stratification. The 679 curated boards that survive the compatibility pass of Appendix O.4 are summarized by a characteristics table with one row per board, recording its net, component, pad, and copper-layer counts. Rows are sorted by pad count in non-decreasing order, so row order coincides with increasing routing difficulty. The pad count bounds the number of terminals a routing method must connect and correlates with both the net count and the total route length. We stratify this sorted table with two parameters and no random component. D3-A (Small) is the 100-row prefix of the table, which corresponds to the boards with at most 31 pads; among the remaining 579 boards, those with at most 100 pads form D3-B (Medium, 287 boards) and those with more than 100 pads form D3-C (Large, 292 boards). The three splits are pairwise disjoint and together cover all 679 boards. Figure 30 shows the resulting pad-count distribution together with both boundaries; Table 2 reports the per-split net, pad, and layer ranges.

Evaluation subsets. Zero-shot evaluation uses a fixed subset of each split rather than the full split, because per-board cost is dominated by the LLM agents and grows steeply with board size. A single D3-A episode already averages 231.1 s and 424,879 tokens for GPT-5.4 (Table 26), and the RL agents run three to six times slower per episode on D3-B than on D3-A (Table 24). For D3-A we evaluate 99 of the 100 boards: `0096_karabas-nano_wifi_revA` is excluded because it cannot be loaded through the engine interface, and it is the only board in the split for which this occurs. For D3-B and D3-C we select ten boards each with the following deterministic procedure, applied identically to both splits:

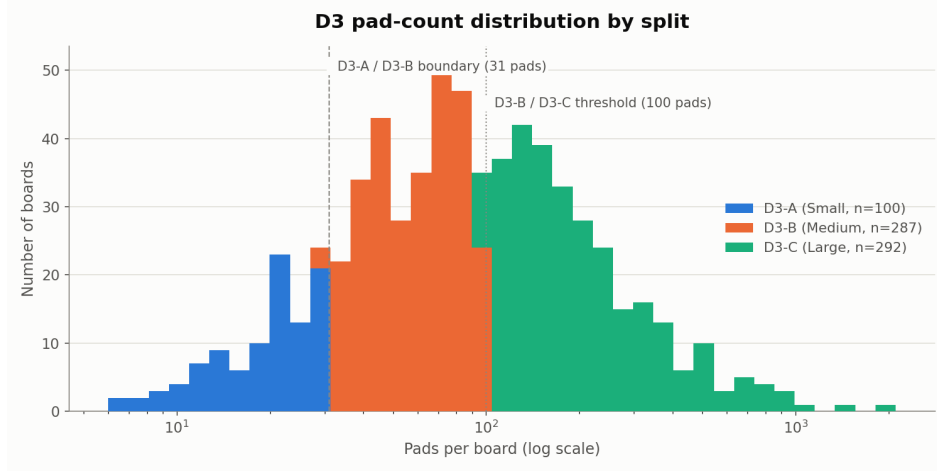


Figure 30: Pad-count distribution of the 679 curated D3 boards, stacked by split (log-scaled horizontal axis). The dashed line marks the D3-A/D3-B boundary induced by the 100-row prefix of the pad-sorted characteristics table (31 pads); the dotted line marks the 100-pad D3-B/D3-C threshold.

Bin	D3-B (Medium)	D3-C (Large)
1	0113_maytal_Maytal	0400_laptimer58_Chickadee
2	0144_ottawa-badges-2016_ottawa-badge-tagger-2016	0423_induction-hob_temperature-sender
3	0174_sensorboard_DiffIR	0448_uedaino_uedaino
4	0203_MOD-MPU9150_mod-mpu9150	0473_data-manager_data-manager
5	0232_ATTiny461Breakout_ATTiny461DevBoard	0496_kitspace_f-91w
6	0260_NiMH-Charger_NiMH_Charger	0520_RGBMatrixPanelCPLD-PhotonBackpack_RGBMatrixPanel_CPLD_negative
7	0288_raspberrypi-3-usb-hub_usb_hub	0546_ozinverter_ozinverterkicad
8	0316_kicad-workshop_fancyboard	0571_Brushless_ESC_Brushless_ESC
9	0344_mavbridge_mavbridge	0599_RGBMatrixPanelCPLD-PhotonBackpack_RGBMatrixPanel_CPLD
10	0376_cat-trainer_tensy_base_pcb	0626_TOBS_HybridChargeController

Table 28: Ten-board evaluation subsets of D3-B and D3-C, one board per quantile bin, ordered by increasing pad count. All twenty boards are two-layer by construction. D3-B is the subset used in the main results; the D3-C subset is fixed for reference and is not evaluated in this paper.

- (1) keep only the two-layer boards, so that every selected instance is handled by all baseline routers and stays comparable to the two-layer D2 boards on which the RL policies are trained;
- (2) for D3-C only, discard the largest 10% of the remaining boards by pad count, whose pad counts reach into the thousands (Table 2) and would otherwise place a single extreme outlier in the top bin;
- (3) sort the survivors by (pad count, net count, board identifier), which makes the order total and independent of file-system enumeration;
- (4) partition the sorted sequence into ten equal-size quantile bins;
- (5) take the lower-median board of each bin.

The procedure involves no random component: the bins and their medians follow from the characteristics table and the parameters above. The ten selected boards therefore span the pad-count range of their split instead of concentrating at its mode. Figure 31 shows the pad-count distribution of the two pools the subsets are drawn from, together with the selected boards, and Table 28 lists them.

O.6 Uses

The intended use is benchmarking PCB routing methods under a shared engine-grounded protocol, including studies of closed-loop tool use, action masking, synthetic-to-real transfer, and constrained sequential decision-making. The dataset should not be used as fabrication signoff: it does not replace professional checks for signal integrity, thermal behavior, EMI, or product-specific manufacturing requirements.

O.7 Distribution

The synthetic datasets are released under the BSD-3-Clause license together with their generator and evaluation code. Curated open-source D3 boards retain the licenses of their source repositories and are distributed with attribution metadata where redistribution is permitted. Each instance is versioned together with the corresponding PCBWorld environment release.

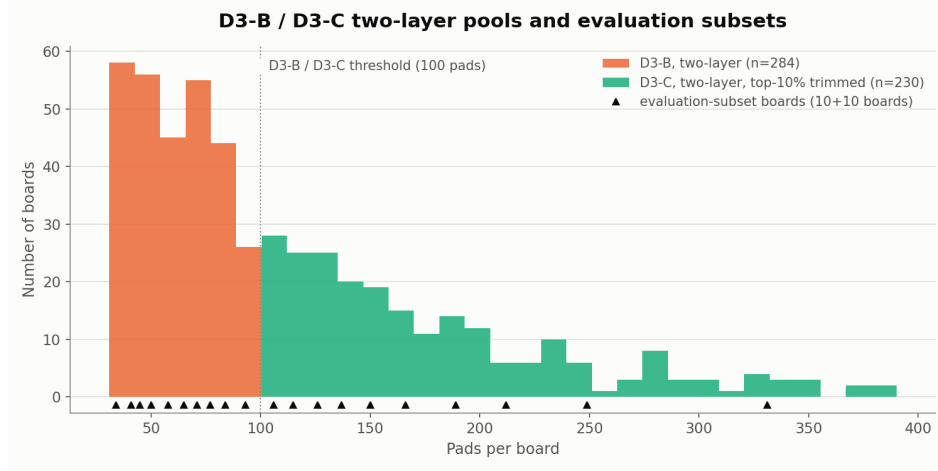


Figure 31: Pad-count distribution of the two-layer D3-B and D3-C pools from which the evaluation subsets are drawn, with the D3-C pool trimmed of its largest 10%. The dotted line is the 100-pad threshold separating the two pools. Triangles below the axis mark the twenty selected boards, one per quantile bin, showing that they cover the pad-count range of each pool.

O.8 Maintenance

Dataset and environment versions are maintained together so that results can be reproduced against the exact board files and evaluator used. Maintainers will preserve attribution metadata, keep prior versions available when practical, and accept issue reports for corrupted files, license metadata problems, incompatible boards, and evaluator discrepancies.

P License, hosting, maintenance

Licensing and versioning. The PCBWORLD repository is released under the BSD-3-Clause license, covering the environment, the synthetic datasets and their generators, the baselines, and the evaluation code. The routing engine derived from KiCAD is distributed separately as PCBWorld-Engine under the GPL-3.0 license, and the PCBWORLD repository pins it as a submodule. Open-source boards retain the license of their source repository and ship with an attribution table for redistribution. Every instance carries a dataset version tag in 1:1 correspondence with the environment version, and each release is pinned to KiCAD 9.0.8 with `ki-cad-python` 0.6.0. The datasheet is provided in Appendix O.

Hosting and maintenance. The environment, PCBWORLD-BENCH datasets, baselines, and evaluation harness are publicly available at <https://github.com/LGAI-Research/PCBWORLD>. The authors will maintain the release, tracking PCBWORLD-BENCH versions against engine versions as described above.